

PAPER • OPEN ACCESS

Integrated error-suppressed pipeline for quantum optimization of nontrivial binary combinatorial optimization problems on gate-model hardware at the 156-qubit scale

To cite this article: Natasha Sachdeva *et al* 2026 *New J. Phys.* **28** 064506

View the [article online](#) for updates and enhancements.

You may also like

- [Performance guarantees of light-cone variational quantum algorithms for the maximum cut problem](#)
Xiaoyang Wang, Yuexin Su and Tongyang Li
- [An expressive ansatz for low-depth quantum approximate optimisation](#)
V Vijendran, Aritra Das, Dax Enshan Koh et al.
- [Evaluation of QAOA based on the approximation ratio of individual samples](#)
Jason Larkin, Matías Jonsson, Daniel Justice et al.



PAPER

OPEN ACCESS

RECEIVED
25 March 2026REVISED
4 May 2026ACCEPTED FOR PUBLICATION
21 May 2026PUBLISHED
3 June 2026

Original content from
this work may be used
under the terms of the
[Creative Commons
Attribution 4.0 licence](#).

Any further distribution
of this work must
maintain attribution to
the author(s) and the title
of the work, journal
citation and DOI.



Integrated error-suppressed pipeline for quantum optimization of nontrivial binary combinatorial optimization problems on gate-model hardware at the 156-qubit scale

Natasha Sachdeva^{*} , Gavin S Hartnett , Smarak Maity , Samuel Marsh, Yulun Wang , Adam Winick, Ryan Dougherty , Daniel Canuto, You Quan Chong, G Adam Cox , Michael Hush , Pranav S Mundada , Christopher D B Bentley, Michael J Biercuk  and Yuval Baum 

Q-CTRL, Sydney, NSW Australia, and Los Angeles, CA, United States of America

^{*} Author to whom any correspondence should be addressed.

E-mail: natasha.sachdeva@q-ctrl.com

Keywords: quantum optimization, QAOA, variational quantum algorithms, combinatorial optimization

Abstract

We introduce a novel hybrid quantum–classical variational optimization method for unconstrained binary combinatorial optimization problems on gate-model quantum computers, integrating a custom variational ansatz, staged feedback-based dual variational parameter update strategies, efficient parametric compilation, automated error suppression during hardware execution, and scalable $O(n)$ classical post-processing to correct for bitflip errors. Without this integrated approach, we show that standard circuit execution at scale produces output indistinguishable from random sampling, establishing the necessity of each pipeline component. We benchmark the method on IBM superconducting quantum computers for several optimization problems drawn from NP-hard problem classes, where the optimization is conducted on hardware with no use of classical simulation or prior knowledge of the solution. For Max-Cut on random regular graphs with topologies not matched to device connectivity, the method achieves approximation ratios (AR) of 100% for unweighted 3-regular graphs up to 156 nodes, weighted regular graphs up to 80 nodes, and weighted 7-regular graphs up to 50 nodes. Applied to higher-order binary optimization, the method finds the ground state energy of 127- and 156-qubit spin-glass models matched to device topology with linear, quadratic, and cubic interaction terms, achieving ARs of at least 99.5% across all instances tested. The method consistently outperforms a classical local solver across all problems. Where published results on identical problem instances are available, our method demonstrates competitive or superior performance. These results demonstrate that an appropriately engineered variational approach enables gate-model quantum computers to produce high-quality solutions for binary optimization problems drawn from NP-hard problem classes at the 156-qubit scale, where naïve implementations are insufficient for good performance.

1. Introduction

Quantum optimization continues to be a central motivation for quantum computing, with industry-relevant combinatorial optimization problems arising across logistics, finance, transportation, networking, and other industries [1]. Although long-term progress toward fault-tolerant devices is expected to unlock the most rigorous quantum speedups, understanding how optimization algorithms perform on today's noisy hardware remains an active and scientifically important area of research. Noisy intermediate-scale quantum era [2] algorithm development enables systematic exploration of hardware-efficient circuit structures, hybrid quantum–classical optimization strategies, and error-aware execution techniques, providing insight into both the opportunities and limitations of near-term devices. Such investigations help shape the design of future fault-tolerant algorithms, refine resource estimates,

and identify algorithmic principles that remain valuable across hardware regimes. Consequently, pushing the scale and robustness of quantum optimization on existing quantum processors is a meaningful step toward practical quantum advantage and continues to motivate the development of new hybrid methodologies.

Many important optimization tasks remain classically challenging at moderate scales—typically hundreds of binary variables with dense interactions or higher-order terms. Widely used solvers such as CPLEX [3] and Gurobi [4], along with more recent specialized heuristics [5], represent decades of algorithmic engineering and provide state-of-the-art performance for large classes of quadratic or linearly constrained optimization problems. Although these solvers are, in principle, exact and capable of certifying global optimality, achieving such guarantees can require substantial computational resources for combinatorially complex instances. As a result, they are frequently operated under practical time or memory limits, where the emphasis shifts from provable optimality to high-quality feasible solutions. Their effectiveness further diminishes when moving beyond quadratic formulations, when constraints induce strong coupling, or when higher-order interactions create rugged energy landscapes. For medium-scale instances on the order of 10^2 – 10^3 binary variables, these factors can lead to solutions that remain far from optimal within acceptable run times, as observed in real-world applications such as periodic train scheduling, where solvers operating under acceptable run-time budgets can under-perform the true optimal solutions by up to 50% [6], and multi-attribute vehicle routing, where rich constraint structures and large search spaces similarly challenge state-of-the-art heuristics [7]. These limitations motivate continued exploration of alternative computational paradigms, including quantum approaches.

Quantum optimization provides a framework for representing both quadratic and higher-order binary problems as Ising Hamiltonians suitable for gate-model and annealing-based devices. A key objective is to develop algorithmic techniques that make effective use of today's hardware while remaining scalable to larger processors as they become available. Although recent hardware roadmaps suggest a more gradual trajectory toward devices with several hundred to a thousand high-fidelity qubits, resource analyses indicate that systems in this regime may provide performance beyond that of optimized classical heuristics for certain problem classes [8–11]. Consequently, evaluating the behavior of quantum optimization algorithms on present-day processors offers valuable insight into their scalability and robustness.

Gate-model quantum computers can execute quadratic unconstrained binary optimization (QUBO) problems and facilitate direct incorporation of higher-order optimization terms and constraints [12–17]. However, the generality of the hardware introduces numerous error pathways that significantly curtail useful system size and performance. The quantum approximate optimization algorithm (QAOA) [18] provides a natural variational framework for these problems, but achieving high-quality solutions on hardware has proven challenging. On superconducting qubit devices, QAOA applied to problems with non-hardware-native graph topologies has produced lackluster results at scales up to 40 qubits [19–22], with standard implementations returning output that is often indistinguishable from random sampling. In trapped-ion processors, successful QAOA implementations capable of returning optimal results have reached approximately 32 qubits, although in [23], this required classical simulation to pre-calculate optimized circuit parameters and relied on high-depth circuits ($p \geq 10$). A full on-hardware optimization of $p = 2$ QAOA for 32-node Max-Cut returning the optimal cut was demonstrated in [24], and a qubit-reuse strategy on 20–32 qubit trapped-ion devices has extended QAOA demonstrations to 130-nodes, although without finding optimal solutions [24, 25].

More recently, several works have expanded the landscape of quantum optimization. In [10], the authors performed noiseless simulations of QAOA on the low-autocorrelation binary sequences problem with up to 40 qubits, finding that QAOA combined with quantum minimum finding (QMF) exhibited more favorable runtime scaling than the best known classical heuristics, contingent on a fault-tolerant implementation of the QMF subroutine; QAOA alone did not show this advantage. They also demonstrated experimental progress in executing the algorithm on Quantinuum trapped-ion processors using algorithm-specific error detection. In [26], the authors introduced a qubit-efficient encoding that maps $O(n^k)$ binary variables onto n qubits, achieving approximation ratios (ARs) estimated to exceed the computational hardness threshold of 0.941 for Max-Cut instances with $m = 2000$ variables on 17 trapped-ion qubits. In [27], the authors showed that fixed linear-ramp QAOA schedules achieve competitive performance without per-instance parameter optimization, demonstrating the protocol on multiple QPU platforms with up to 109 qubits and circuits requiring over 21 000 CNOT gates. In a separate cross-platform benchmarking study [28], QAOA-based benchmarks were evaluated across 19 QPUs from five vendors, finding that Quantinuum H2-1 maintained coherent computation on a fully connected 56-qubit Max-Cut instance. These advances collectively illustrate both the rapid evolution of the field and the diversity of approaches from qubit-efficient encodings that maximize problem size per physical qubit, to methods

that eliminate the classical optimization loop, to brute-force depth scaling on hardware with sufficient coherence.

Quantum annealing (QA) provides an alternative approach to binary optimization [12, 29] in which the limited functionality of the hardware reduces error pathways and simplifies device scaling. While gate-model quantum computers and quantum annealers share formal equivalence from the perspective of computational complexity [30], practical comparisons have generally favored annealers for QUBO problems [31, 32]. Higher-order terms and/or constraints have also been included by leveraging QA scaling advantages via the use of slack variables and penalty terms [33–35]. In direct comparisons between standard QAOA implementations on gate-model hardware and quantum annealers, annealers have typically delivered superior solutions [36–38]. In [36], the authors confirm this pattern at the 127-qubit scale on *ibm_washington*, finding that short-depth QAOA ($p=1,2$) was outperformed by D-Wave quantum annealers across all higher-order Ising model instances examined, which were problems where the connectivity matched the gate-model device’s native heavy-hex topology. A central question, then, is whether this performance gap reflects a fundamental limitation of gate-model optimization or whether it can be bridged through more careful integration of algorithmic design with hardware error management. We investigate these problem instances, along with Max-Cut on random regular graphs, in this work.

We introduce an integrated hybrid quantum–classical optimization pipeline for binary combinatorial optimization problems on gate-model quantum computers that returns optimal or near-optimal solutions up to whole-chip 156-qubit implementations. We provide an overview of the quantum and classical components of the algorithm, including the use of a novel variational ansatz, a dual variational parameter update strategy using measurement-based feedback, and efficient classical post-processing to address bitflip errors. We benchmark this solver on IBM quantum computers using Q-CTRL parametric compilation and error suppression for several unconstrained binary optimization problems and compare against published results using alternate hardware systems. First, we consider Max-Cut instances [39] for random regular graphs up to degree-7. Our implementation returned the correct Max-Cut value over all instances tested up to 156 nodes which were mapped directly to 156 qubits. We provide a quantitative comparison to previously published trapped-ion results using identical problem instances [23]. Next, we applied the algorithm to higher-order unconstrained binary optimization (HUBO) problems, searching for the ground state energy of device-spanning random-bond spin-glass models with linear, quadratic, and cubic terms following [35]. For a 127-qubit problem, we demonstrated the ability to find the correct energy minimum in four of six instances tested, including an instance where the annealer did not find the correct ground-state energy; for the two additional instances tested, solution ARs exceeded 99.5%. For a 156-qubit problem, our algorithm found the global minimum on all instances considered. For both problem types, the algorithm outperformed a heuristic local solver that applies a greedy optimization to a random distribution of candidate solutions. Nonetheless, these problems are modest enough in scale such that they may be solved exactly using classical methods; we used CPLEX to provide the ground-truth. We do not claim quantum advantage over state-of-the-art classical solvers for the problem instances treated in this work; the primary objective is to demonstrate reliable quantum optimization performance at a scale where standard implementations fail. Overall, these results represent some of the largest quantum optimizations returning optimal solutions performed to date with on-device angle optimization for two problem classes that challenge classical solvers.

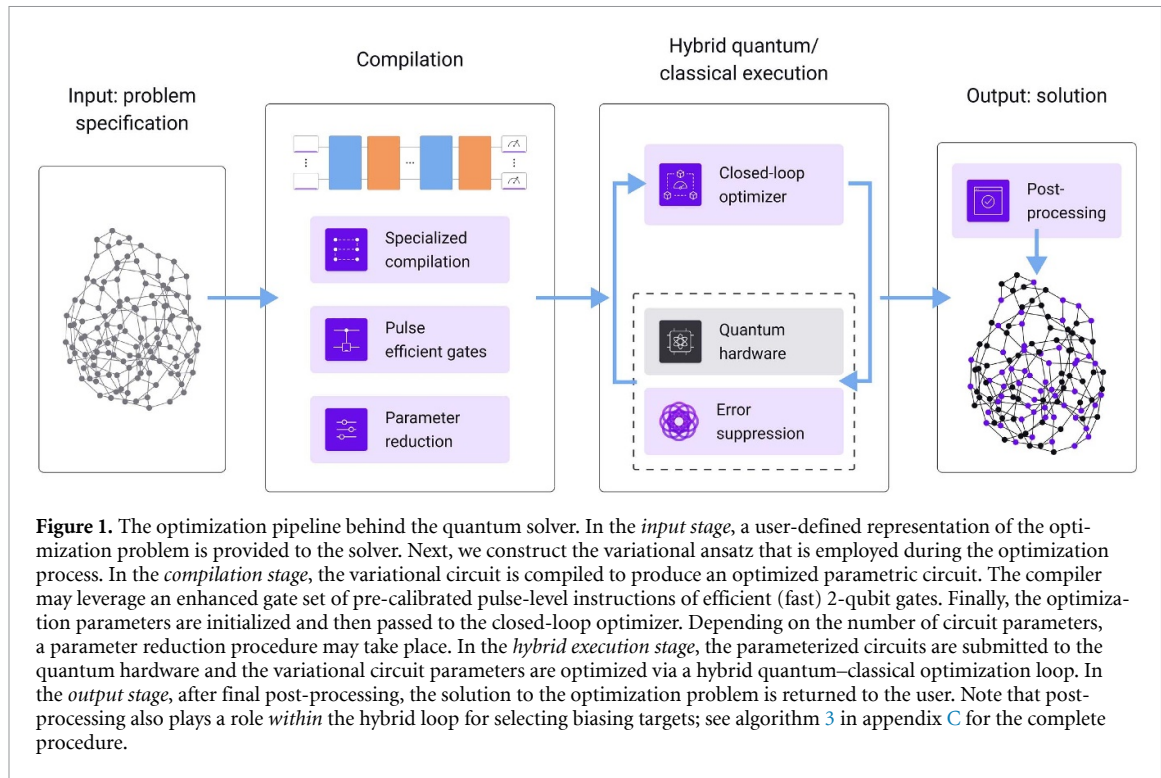
2. Integrated pipeline for quantum optimization

We introduce a hybrid quantum–classical algorithm for unconstrained binary optimization problems over length- n bitstrings of the form

$$\min_{\mathbf{x} \in \{0,1\}^n} C(\mathbf{x}), \quad (1)$$

where the objective function $C(\mathbf{x})$ may be represented as a polynomial over the binary variables. The key elements of this algorithm, combined with Q-CTRL’s compilation and error suppression tools, are visually represented in figure 1. The algorithm builds upon on a commonly used quantum optimization algorithm, the QAOA, which is a hybrid classical-quantum algorithm inspired by the adiabatic theorem to approximate solutions for combinatorial optimization problems [18].

After defining the optimization problem as in equation (1), the variational circuit is constructed as a standard QAOA implementation with cost and mixer Hamiltonians parameterized by γ_i and β_i where $i = 1, \dots, p$ and p represents the number of QAOA layers used. However, a change is made to the initial



state, using additional parameterization that deviates from the initial layer of Hadamard gates in standard QAOA and replacing them with $R_y(\theta_j)$ gates, where $j = 1, \dots, n$ represents the qubit index. All θ_j angles are initialized to $\pi/2$, recovering the standard QAOA ansatz at the beginning of the optimization procedure.

The parameterized circuits are iteratively submitted to the quantum hardware as part of the hybrid quantum–classical optimization loop in which only the γ_p and β_p QAOA parameters are optimized with a closed-loop classical optimizer using the results from quantum circuit executions in order to iteratively minimize the cost function. The initial state parameters are updated at fixed points during the optimization process as detailed below to bias the initial state toward the best solution found over prior iterations. Execution of each circuit is performed using Q-CTRL’s parametric compilation and error suppression pipeline detailed in [40] in order to reduce gate-level and circuit-level errors. Finally, we post-process the measured distributions with a simple $O(n)$ greedy optimization pass. We review the key elements of the optimization algorithm pipeline in detail below.

Modified variational ansatz: We aim to overcome the challenge of limited accessible circuit depth in commercial quantum processors, as this curtails the achievable QAOA repetition level, denoted p , well before the value $p > 11$ anticipated to be required (at a minimum) to approach definitive quantum advantage [8, 41] in a conventional implementation. We follow previous literature to address both scalability and generalizability [11] through use of a modified QAOA ansatz and a dual variational parameter update strategy.

The standard QAOA ansatz starts with an initial n -qubit equal superposition state established by application of a Hadamard gate to each qubit, followed by alternating layers of unitary transformations corresponding to the cost and mixer Hamiltonians. The cost Hamiltonian, H_C , encodes the optimization problem to be solved and the corresponding unitary transformation, U_C , is parameterized by variational parameters γ_i , where $i = 1, \dots, p$. The mixer Hamiltonian, H_M , facilitates transitions between states in order to explore the solution space, and the mixing unitary U_M for each layer is tuned with the variational parameters β_i .

Our modified QAOA ansatz, shown in figure 2, substitutes the typical equal-superposition starting state with a new input state generated using arbitrary rotation operators $R_y(\theta_j)$, where $j = 1, \dots, n$ and θ_j varies individually for each qubit. This parameterization is used to encourage convergence for standard QAOA by gradually biasing the optimization search space toward low-cost solutions found during the optimization process. All optimizations start with all θ_j set to $\pi/2$, producing the uniform superposition state, which is effectively identical to a standard QAOA ansatz. Our approach varies in that

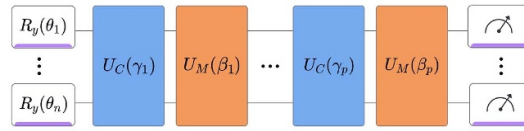


Figure 2. The ansatz we employ for solving unconstrained optimization problems. The initial state is parameterized angles θ_j , for $j = 1, \dots, n$, followed by the standard QAOA repetition pattern of a cost-unitary and a mixing-unitary, defined by the γ_i, β_i parameters for $i = 1, \dots, p$. This ansatz has a total of $n + 2p$ parameters where n is the number of qubits and p the number of QAOA layers.

at specific intervals during the optimization process, we use this parameterization to bias the initial state toward the best solution observed during previous iterations. This adaptive approach builds on related work, but constitutes a departure from warm-start QAOA methods [42–45] in which the initial state remains unchanged throughout the circuit optimization. Instead, our ansatz starts from the zero-information point in the first cycle of the classical loop, and the initial state parameters are updated two to three times during the optimization process, while $\{\gamma, \beta\}$ are optimized in every iteration as in a typical application of QAOA. The additional parameterization in this ansatz and parameter update strategy enables the solver to converge on correct solutions without the need for large p by reducing the solution space through biasing at select intervals during the optimization. Such an approach mitigates practical concerns around the impact of incoherent errors and limited circuit depths encountered on today’s machines. This feedback-based parameter update strategy may also be viewed through the lens of quantum optimal control theory [46, 47], where measurement-driven updates to variational parameters parallel control-theoretic approaches to steering quantum dynamics. More details are provided in appendix C.

Efficient parametric compilation: Our solver employs an efficient form of parametric compilation in order to reduce execution overheads—a practical rather than fundamental consideration that can have substantial impact on the achieved solution quality. During operation of the solver, the parameters of the executed quantum circuits are updated in real-time based on feedback from the classical optimizer during execution of the hybrid loop. This procedure repeats itself anywhere from tens of times for small problems to thousands of times for more complex problems. Compilation overhead causes effective dead-time in which the hardware system is not being sampled, introducing sensitivity to drifts on the timescale of the optimization loop’s execution. Accordingly, inefficient compilation introduces additional sensitivity to errors, the cumulative effect of which can impair convergence.

Our parametric compilation strategy incurs the initial compilation overhead only once, followed by real-time updates of the circuit parameters during the execution of the hybrid loop to minimize dead-time. Successful implementation requires careful consideration of the fact that symmetries and circuit identities that apply to specific parameters do not hold in general [48]. Additionally, while the available native gate set is universal, commonly used ansätze such as that in figure 2 may be compiled more efficiently by enriching the available gate set. In particular, our compiler can leverage pulse-level instructions of pre-calibrated gates to realize faster and higher quality arbitrary two-qubit rotations [49].

Automated error suppression in hardware execution: The solver uses an automated error-suppression pipeline [40] during hardware execution of each parameterized circuit. This strategy combines dynamical decoupling embedding for simultaneous cross-talk and dephasing suppression [50] and automated AI-driven gate-waveform replacement [40]. This pipeline has been shown to deliver substantial performance enhancements—exceeding $1000\times$ in algorithmic benchmarks such as Bernstein–Vazirani and quantum volume on IBM hardware [40, 51]. Within the present context, figures 4(a)–(c) demonstrates the impact of error suppression on a representative 40-qubit QAOA circuit. In this work, we focus on the use of the modified variational ansatz and our results for two key classes of unconstrained optimization problems. We note that the modified variational ansatz, dual parameter update strategy, biasing analysis, and classical post-processing are fully described in this manuscript and may be independently reproduced; the parametric compilation and error-suppression components rely on the externally developed tools referenced above.

Automated classical optimization loop: We implement the classical component of the optimization loop using a covariance matrix adaptation evolution strategy (CMA-ES); it is a fast, stochastic, and derivative-free method used for non-convex numerical optimization problems [52]. We select conditional value-at-risk (CVaR) as the objective function, as it has been shown to lead to faster convergence for

combinatorial optimization problems [53]. CVaR provides a weighted average of the tail end of a distribution; it is evaluated for the results of each circuit execution and CMA-ES uses these values to define the parameters for the variational circuits of the subsequent optimization step. The CVaR α parameter is the percentage of the tail used. For all data presented in this work, $\alpha = 0.35$ was used (see appendix C for justification).

To maximize the efficiency of quantum-resource utilization, we use multiple parameter update strategies in the optimization procedure. The number of variational parameters (the $2p$ parameters γ and β) is reduced by employing the Fourier parameterization introduced in [54], although for the problems presented in this work $p = 1$ was sufficient to find the known correct solutions (in which case, the Fourier parameterization is equivalent to the standard parameterization). Additionally, the θ angles are initialized to $\pi/2$ (recovering the standard QAOA ansatz) and are updated a maximum of three times during the optimization run according to the information collected during the previous optimization steps; this procedure takes advantage of the bitstring distributions observed in the prior optimization steps to select the lowest-cost bitstring observed and bias the initial state toward it, with progressively stronger biasing strength. Every time the θ angles are updated, we include a $(\gamma, \beta) = (\mathbf{0}, \mathbf{0})$ circuit, equivalent to a $p = 0$ circuit, as a baseline comparison (see appendix C for details). The γ and β parameters are initialized randomly; this method does not require problem-specific parameter initialization in order to converge toward high-quality solutions [55].

Classical post-processing: Uncorrelated bitflip errors in the measured output distribution significantly impact solution quality. These errors can come from a variety of sources, commonly dominated by device T_1 noise and measurement errors. For large-scale problems beyond 100 qubits, eliminating these errors completely during circuit execution is unrealistic with contemporary devices. However, these errors may be addressed by a purely classical post-processing step.

We implement an $\mathcal{O}(n)$ naïve greedy optimization on the solution bitstrings returned from measuring the final, optimized variational circuit. This may be done efficiently given that the cost function C is local—evaluating the cost difference $C(\mathbf{x}') - C(\mathbf{x})$ for $\mathbf{x}, \mathbf{x}'$ differing by a single bitflip involves a constant number of steps. With no additional measurements or circuit modifications required, this procedure randomly traverses the measured solution bitstring and greedily flips bits if they improve the bitstring cost. This approach continues until no improvement is realized from successive bitflips, and we cap the number of full traversals of the bitstring to five passes. This implementation is not problem-dependent and can be applied to any unconstrained binary combinatorial optimization problem. Importantly, it requires no additional execution overhead in increased shots on hardware. In our pipeline, this greedy post-processing serves a dual role: (i) during the optimization loop, it is applied to sampled bitstrings to select the target $\mathbf{b}_T$ for initial-state biasing updates (see appendix C), and (ii) after the final circuit execution, it refines the output distribution to correct for uncorrelated bitflip errors. The CVaR objective used for (γ, β) optimization is always evaluated on error-suppressed samples prior to greedy post-processing. The use of greedy classical optimization in conjunction with QAOA has been explored in references [56–60].

This post-processing pass is not specifically a bitflip error-correcting pass but a computationally cheap optimization to provide some level of robustness to uncorrelated bitflip errors that are unavoidable on current noisy devices, especially in large-scale problems.

3. Hardware implementation

We test the performance of the solver via execution on the 127-qubit IBM quantum computers *ibm_brisbane* and *ibm_sherbrooke*, as well as the 156-qubit *ibm_kingston* and *ibm_boston* devices. The data on 127-qubit IBM Eagle devices was collected in 2024, and the data on 156-qubit IBM Heron devices was collected in September 2025 (Max-Cut) and January 2026 (spin-glass instances). In all cases, the circuit executions within the full optimization loop are executed on the quantum device, without any use of classical simulation to optimize parameters. The method implementation is not tuned or modified for the specific problems we implement aside from the hardware-efficient spin-glass problem embedding, and the results we show faithfully represent the quality of expected solutions for any optimization problem in the same class as the ones we optimized.

We evaluate performance by determining a series of metrics which highlight the quality of the output solutions relative to the global minimum (or minima, in the case of degenerate solutions), and the likelihood of achieving the correct solution. In our tests, the global minimum is determined using the classical solver CPLEX [3]. First, we evaluate the AR; for a solution defined over binary variables as

$\mathbf{x} \in \{0, 1\}^n$, the AR is

$$\text{AR}(\mathbf{x}) = \frac{C(\mathbf{x}) - C_{\min}}{C_{\min} - C_{\max}}, \quad (2)$$

where $C(\mathbf{x})$ is the objective function, $C_{\min}$ and $C_{\max}$ are the minimum and maximum objective value of the problem (both may be obtained via CPLEX, the maximum corresponds to minimum objective for the negated problem $-C(\mathbf{x})$). An $\text{AR} = 100\%$ corresponds to the algorithm finding a solution matching the minimum.

In addition, we provide standard metrics allowing quantitative comparison of solution quality by including the likelihood that the top solution is returned. This metric is particularly relevant for cases where the achieved $\text{AR} = 100\%$, i.e. the top solution is the correct solution, as it provides additional information on the stability of the optimization process and the relative correctness of each sampling of the optimized circuit. Any circuit, when sampled, will return a distribution of outputs that ideally skews towards the ground truth; a high-quality solver will deliver a large overlap of outputs with the ground truth while a weaker solver may find the ground-truth solution only rarely if at all.

When available, we compare the quality of solutions returned from the solver against published results derived from a commercial D-Wave annealer and a Quantinuum trapped-ion gate-model quantum computer. In such instances, the problems we execute are defined identically to those executed in the relevant references. In addition, to provide insight into the relative challenge of the test problems we explore, we also include a classical local solver in our benchmarks. A summary of all results achieved with comparisons against previously published literature appears in table 1. Additional information including estimates of execution time and cost is included for completeness in appendix A.

3.1. Max-Cut

The Max-Cut problem is a classic problem in combinatorial optimization and graph theory. It involves partitioning the vertices of a graph into two distinct subsets such that the number of edges between the two subsets is maximized. There is a formal equivalence between Max-Cut and QUBO problems—any Max-Cut instance may be mapped to a QUBO instance and vice versa [62]. Max-Cut on k -regular graphs, meaning graphs where each node is connected to exactly k other nodes, are NP-hard problems in the case where $k > 2$ and have real-world significance, particularly in networking [39, 63]. For arbitrary graphs, this problem is also challenging because it does not exploit native symmetries or connectivity of the underlying quantum hardware in use. Accordingly, Max-Cut on random 3-regular graphs is a standard benchmark problem used to test the efficacy of quantum algorithms against classical counterparts by being both a simple and nontrivial example of a QUBO problem.

Max-Cut can be framed as a minimization of an objective function of the form

$$C(\mathbf{z}) = -\frac{1}{2} \sum_{i < j} A_{ij} (1 - z_i z_j). \quad (3)$$

Here, we have expressed the cost function in terms of the Ising spin variables $\mathbf{z} = 2\mathbf{x} - 1 \in \{-1, 1\}^n$, and A is the $n \times n$ adjacency matrix of the graph ($A_{ij} \neq 0$ if $(i, j) \in G$, otherwise $A_{ij} = 0$), with n is the number of nodes. We distinguish between unweighted graphs where $A_{ij} = 1$ if $(i, j) \in G$, and weighted graphs where $A_{ij} = w_{ij} \in \mathbb{R}$ if $(i, j) \in G$. For all problems we present, the number of nodes is equivalent to the number of qubits used on the device. Candidate partitions of the graph are then represented by length- n bitstrings (entries are zero or one depending on which subgraph each node is assigned to), and each graph node is assigned to a single qubit. Given a partition, the cut value is calculated straightforwardly as the number of edges between the two groups.

In figures 3(a) and (b), we demonstrate the performance of our solver on two example Max-Cut problems: An unweighted 156-node 3-regular graph and a weighted 80-node 4-regular graph (see table 1 for summary results on these and additional instances). In these figures, we plot the histogram of cut values calculated from the bitstring solutions returned by the solver, output as measurements of the optimal circuit. A good solution approaches and ideally overlaps the actual maximum value derived from CPLEX (denoted by a dashed vertical red line) towards the right side of the plot.

Our performance baseline is set by sampling bitstrings uniformly at random (light gray histogram), akin to a random brute-force scan through the solution set. In all cases, the number of samples is taken to be the same as the samples of the optimal QAOA circuit. We choose this baseline as an indication of the difficulty of the problem and also because, as shown later, a naïve QAOA implementation using standard Qiskit compilers returns results indistinguishable from random selection (see figure 4 for detail).

Table 1. Comparison between different solvers on a variety of (top) Max-Cut problems and (bottom) spin-glass energy minimization problems. Max-Cut problems are specified by four parameters $(n, k, s, u/w)$, where n is the number of graph nodes, k , the degree of the regular graph, s , the seed of the random graph and u/w indicating if the graph is unweighted (u) or weighted (w). The seeds for the first three problems define the problems accessed from [64]. The other Max-Cut problems are defined by the seeds used to generate them with NetworkX's random regular graph generator [65]. The spin-glass problems are specified by instance numbers. For the first 6 entries, these correspond to the same instance numbers used in [61] for the problems generated for the *ibm_washington* device (note that in this work, these problems were executed on other devices). For the last 4 entries, the instance number corresponds to the random seed used to generate the problem instance. We use the same circuit construction approach as [66] for the spin-glass problems to facilitate a fair comparison. Exact global minimum and maximum values were evaluated using CPLEX. For each solver, we list the best objective found (top solution), its AR, and the observed top solution likelihood (estimated as the ratio of the number of times it was sampled to the total number of samples). The integrated solver data indicates the top solution found out of N_{shots} used for sampling the optimal circuit. For all spin-glass problems $N_{\text{shots}} = 15k$, where for the Max-Cut problems the number of shots varies between 6–15k depending on the number of nodes (see appendix E for more experimental details). The IBM device used for each problem is indicated as Br for *ibm_brisbane* (127Q), S for *ibm_sherbrooke* (127Q), K for *ibm_kingston* (156Q), and Bo for *ibm_boston* (156Q). The local solver data indicates the top solution found out of N_{shots} local minima found by a greedy algorithm applied to a random distribution of candidate solutions (see appendix B for implementation details). The QA data, taken from [36], indicates the top solution found out of 1328 000 samples collected from two Pegasus-class D-Wave devices. The Quantinuum data, taken from [64], is based on 1024 samples of a pre-optimized QAOA circuit, executed on the H2-1 device, where for each problem the best performing p , out of $p \leq 11$, is shown. These comparisons involve fundamentally different optimization paradigms: our solver performs full variational optimization on hardware, the Quantinuum results use classically pre-optimized parameters from simulation, and the D-Wave results employ grid search over annealing parameters.

Max-Cut

									Quantinuum (H2-1)		
Global		Q-CTRL (IBM Quantum)				Local solver			Classically opti-		
		Full hybrid optimization							mized in simulation		
Instance	Min/Max	Max	AR (%)	Likelihood	Device	Max	AR (%)	Likelihood	Max	AR (%)	Likelihood
(28, 3, 102, u)	0/40	40	100	9.44×10^{-1}	S	40	100	6.20×10^{-1}	40	100	1.78×10^{-1}
(30, 3, 264, u)	0/43	43	100	9.18×10^{-1}	S	43	100	4.68×10^{-1}	43	100	1.04×10^{-1}
(32, 3, 7, u)	0/46	46	100	8.25×10^{-1}	S	46	100	4.06×10^{-1}	46	100	1.27×10^{-1}
(80, 3, 68, u)	0/106	106	100	1.38×10^{-1}	Br	106	100	1.00×10^{-4}	Not applicable		
(100, 3, 12, u)	0/135	135	100	1.21×10^{-1}	Br	135	100	1.00×10^{-4}			
(120, 3, 8, u)	0/163	163	100	8.59×10^{-2}	Br	163	100	1.67×10^{-4}			
(156, 3, 0, u)	0/213	213	100	2.05×10^{-1}	K	211	94.4	1.33×10^{-4}			
(156, 5, 0, u)	0/324	324	100	2.13×10^{-3}	K	322	95.4	6.67×10^{-5}			
(50, 6, 28, w)	0/73.2	73.2	100	8.10×10^{-2}	S	73.2	100	3.67×10^{-2}			
(50, 7, 27, w)	0/86.25	86.25	100	7.71×10^{-2}	S	86.25	100	3.67×10^{-2}			
(60, 5, 36, w)	0/76.25	76.25	100	1.01×10^{-1}	S	76.25	100	1.57×10^{-2}			
(70, 4, 75, w)	0/83.75	83.75	100	6.16×10^{-2}	S	83.75	100	2.20×10^{-3}			
(80, 4, 46, w)	0/88.0	88.0	100	2.08×10^{-2}	S	88.0	100	2.90×10^{-3}			

Spin Glass

Global		Q-CTRL (IBM Quantum)				Local solver			D-Wave (Pegasus)		
		Full hybrid optimization							Grid search		
Instance	Min/Max	Min	AR (%)	Likelihood	Device	Min	AR (%)	Likelihood	Min	AR (%)	Likelihood
0	−200/192	−200	100	1.34×10^{-4}	S	−188	96.9	6.67×10^{-5}	−200	100	7.53×10^{-7}
3	−198/184	−198	100	1.18×10^{-1}	S	−190	97.9	6.67×10^{-5}	−198	100	7.53×10^{-7}
5	−198/192	−198	100	2.03×10^{-1}	S	−188	97.4	2.67×10^{-4}	−194	99.5	3.77×10^{-6}
10	−202/190	−202	100	1.92×10^{-2}	Br	−194	98.0	6.67×10^{-5}			
11	−180/196	−178	99.5	1.69×10^{-1}	Br	−170	97.3	6.67×10^{-5}			
69	−190/212	−188	99.5	1.75×10^{-1}	Br	−180	97.5	6.67×10^{-5}			
0	−242/242	−242	100	2.17×10^{-1}	K	−226	88.2	1.33×10^{-4}	No published data		
0	−242/242	−242	100	1.33×10^{-4}	Bo	−226	88.2	1.33×10^{-4}			
1	−240/238	−240	100	4.64×10^{-1}	Bo	−228	88.2	6.67×10^{-5}			
2	−250/246	−250	100	1.67×10^{-3}	Bo	−230	87.0	6.67×10^{-5}			

Executing the entire quantum-solver workflow depicted in figure 1 results in a shift of the output histogram to large cut values, and produces solutions which attain the known maximum cut value. Up to 156 qubits, we find that not only does the quantum solver distribution overlap the maximum cut value, but it is asymmetrically skewed towards this value resulting in a high likelihood of achieving the correct answer. The quantum solver also provides superior solutions in these cases to those found using a classical local solver shown in the dark gray histogram (see appendix B for implementation details).

In table 1, we show additional results across different Max-Cut problems with higher-density graphs, up to 7-regular graphs. In all executions attempted, and across all specific problem instances within each

class, the quantum solver achieves $AR = 100\%$. In these cases, the solver also uniformly outperforms the classical local solver in solution likelihood, and matches the CPLEX ground-truth solutions. Additional data across the problem instances explored appear in appendix E.

These results demonstrate superior performance to the previously published results on a trapped-ion quantum processor with 32 qubits in [23]. Because we do not have access to the same trapped-ion device, we cannot comment on relative base-hardware performance had the Q-CTRL pipeline been implemented; rather we only make comparisons to the results published to date. Table 1 provides a direct comparison of executions of the quantum solver on IBM hardware using the same problem instances presented in [23] for smaller graphs up to 32 nodes. Our solver achieves $AR = 100\%$ across all instances for $p = 1$. In execution of higher-depth circuits with $p = 10$ and $p = 11$, the trapped-ion device returned the maximum cut, but with substantially lower likelihood than our solver; across these problem instances our solver finds the maximum cut with likelihood of $\sim 82\%$ – 94% , while on Quantinuum the range was $\sim 10\%$ – 18% . Perhaps more importantly, the trapped-ion demonstrations were limited to hardware execution of a single circuit using pre-optimized γ and β parameters attained via classical numeric simulation. Our solver, by comparison, exclusively runs full hardware execution of the QAOA optimization loop. However, we note that there exists another trapped-ion demonstration of a full optimization of $p = 2$ QAOA for a 32-node Max-Cut problem that returns the optimal cut [24].

3.2. HUBO ising spin glass

Next, we consider an optimization problem designed to find the ground-state energy of a random-bond Ising model possessing cubic terms. This is an example of a HUBO, known to be more challenging to solve than QUBO problems [12–15].

In our benchmarking, we follow the same model as introduced in [66], adapted to two different IBM device architectures. We consider a spin Hamiltonian with 1-, 2-, and 3-body interaction terms tailored to the heavy-hex connectivity of (1) the Eagle-class devices with 127-qubits, and (2) the Heron-class devices with 156-qubits. Denoting the heavy-hex connectivity graph as $G = (V, E)$, the cost function for this problem is

$$C(\mathbf{z}) = \sum_{i \in |V|} a_i z_i + \sum_{(i,j) \in E} b_{i,j} z_i z_j + \sum_{(i,j,k) \in W} c_{i,j,k} z_i z_j z_k. \quad (4)$$

The coefficients a , b , and c represent the 1-, 2-, and 3-body coupling strengths, respectively, with W denoting a specific set of 3-tuples corresponding to 3-body interaction terms optimized for efficient implementation on IBM hardware. For Eagle-class devices, the resulting objective function comprises 127 linear terms, 142 quadratic terms, and 69 cubic terms, while Heron-class devices support expanded problem instances with 156 linear terms, 176 quadratic terms, and 244 cubic terms. The values of the coefficients for each term are randomly chosen to be ± 1 . We refer to different choices of these coefficients as model instances, and use identical instances and notation as in [66] for the problems defined on the *ibm_washington* device topology.

In figure 3(c) we show the performance of our solver for ‘instance 0’ defined on *ibm_kingston* (see appendix E for the remaining instances). Here, the distribution produced by the quantum solver is narrowly concentrated around the edge of the energy band (in this case solutions at the left edge of the plot are best), producing high quality ground-state estimates that match the known ground state energy. As above, this solution outperforms all candidate solutions returned by random sampling. As for Max-Cut, we also benchmark against a local solver, showing that our solver consistently outperforms the local solver. We provide further detail in table 1 where we highlight the performance of the solver over multiple problem instances reproduced from [36]. The worst-case AR returned for the selected problem instances tested was $AR = 99.5\%$.

Through the introduction of additional variables and interaction terms in the energy, this HUBO problem may be converted to a QUBO problem, allowing for execution on quantum annealers [12]. A 127-node HUBO can be written as a 265-node QUBO with 625 quadratic terms and 69 penalty constants. The nonlocal nature of the additional quadratic terms and the additional energy scales introduced by the multiple penalty terms render the extended QUBO problem challenging for quantum annealers [67]. For that reason, in order to benchmark this problem using an annealer, the authors in [36] developed a dedicated order-reduction scheme that was tailored for the specific problem and for the specific QA device used in that work.

In [36], quantum annealing was benchmarked for ten different problem 127-qubit instances. The instance above (number 5) was the only one where the ground state was not sampled at all. Yet, in other instances where $AR = 100\%$ for both the quantum solver and the annealer, success on the annealer

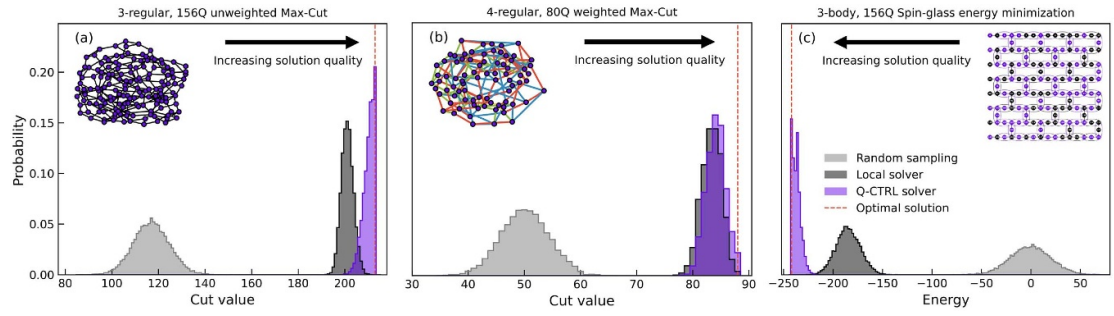


Figure 3. Performance of the quantum solver on three combinatorial optimization problems. (a) A Max-Cut instance for a 3-regular unweighted random graph with 156 nodes. The plot shows the distribution of cut values for 15k configurations sampled from the optimal circuit found by the quantum solver, executed on *ibm_kingston*. The true maximum cut value of 213 is successfully attained by the solver. The random search and classical local solver distributions show the result of 15k configurations sampled uniformly at random and 15k local minima. (b) A Max-Cut instance for a 4-regular weighted random graph with 80 nodes. The weight of each edge was randomly chosen from four possible values $[1/4, 1/2, 3/4, 1]$, represented by different edge colors. Shown, cut values for 10k configurations sampled from the optimal circuit found by the quantum solver, executed on *ibm_brisbane*. The true maximum cut value of 88 is successfully attained by the solver. (c) Energy minimization of a 156-node high-order spin-glass model with parameters defined from *ibm_kingston* instance 0 following similarly defined problems in [61]. The plot shows the energy distribution for 15k configurations sampled from the optimal circuit found by the quantum solver, executed on *ibm_kingston*, and for 15k configurations sampled uniformly at random (brute-force) and 15k local minima (local solver). The lowest and highest energies of this model instance (the energy band edges) are known to be -242 and 242 , respectively. As in (a), (b), the true ground state was attained by the solver.

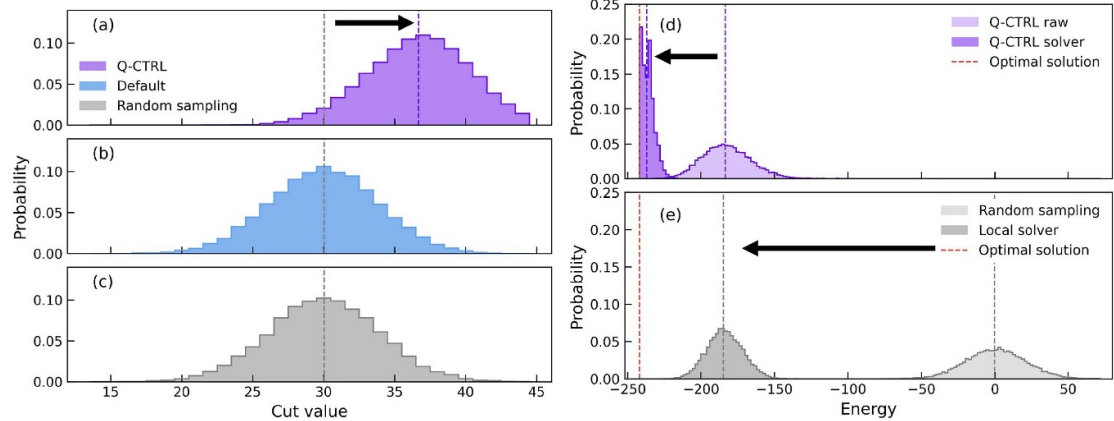


Figure 4. (a)–(c) The role of error suppression. The distribution of cut values obtained by sampling 16 384 configurations from a $p = 1$ QAOA circuit for a Max-Cut instance for a randomly generated 3-regular graph with 40 nodes. The circuit was executed on the *ibm_sherbrooke* device utilizing two performance management schemes, (a) using Q-CTRL error suppression pipeline and (b) Qiskit version 0.44, with optimization level 3 and resilience level 1. In all executions, the equal weights superposition state served as an initial state, optimal QAOA parameters were used and the data shown does not include post processing. (c) A reference distribution corresponding to uniformly sampling bitstrings at random. The purple and gray dashed lines indicate the mean of the Q-CTRL and the reference distribution. (d) and (e) The role of post processing. (d) The distributions of cost values obtained from the optimal circuit returned by our pipeline, before and after the application of post-processing, for cubic spin-glass instance 0 run on *ibm_kingston* also shown in figure 3(c). The (γ, β) parameters were optimized using the CVaR objective evaluated on measured samples prior to greedy post-processing; the post-processing comparison shown here is applied only to the output of the final optimized circuit. (e) For reference, we show the distributions of cut values obtained by 15k random bitstrings and by applying post-processing on each of these 15k configurations, denoted as ‘local solver’.

was marginal. For example, in two instances the correct ground state was measured only once out of 1328 000 samples, corresponding to success likelihood of $\sim 7.5 \times 10^{-7}$ (see instances 0 and 3 in table 1). By contrast for these same instances the quantum solver sampled the solution with likelihood of $\sim 1.3 \times 10^{-4}$ and ~ 0.118 . We note that this comparison involves different optimization strategies: the quantum annealing study employed grid search over annealing parameters, while our approach optimizes variational circuit parameters, making a direct methodological equivalence challenging to establish.

4. Benchmarking performance analysis

The results presented above indicate that it is possible to achieve useful and correct results from quantum optimization executed on gate-model superconducting quantum computers using all available qubits on the device. The ability to consistently solve complex problems at large scales, as we demonstrated, results from the combination of performance enhancements delivered via various parts of the full solver pipeline. Although the results are based on single optimization runs per problem instance, with the cost of quantum processor access (see appendix A) being the primary constraint on replication, the consistency of high ARs ($AR \geq 99.5\%$) across 23 diverse problem instances spanning two problem classes, four quantum devices, and qubit counts from 28 to 156 provides evidence of systematic robustness. The role and contribution of each subroutine strongly depends on the optimization problem; below we include experimental comparisons in order to provide qualitative insights regarding the role of each component.

Information extracted during the quantum circuit execution is the core difference between the quantum solver and classical counterparts. Therefore, the ability to faithfully execute circuits on the quantum hardware is directly translated to the value the quantum solver provides. We now discuss the essential role played by error suppression in ensuring high-quality outputs from the quantum solver. We build on previous demonstrations that the QAOA landscape could show up to $200\times$ greater structural similarity to the ideal when comparing default circuit implementation to that realized in our integrated error-suppressed quantum solver [40].

Figures 4(a)–(c) shows the distribution of sampled circuit outputs for an example 40-qubit Max-Cut problem in which we isolate the hardware execution by removal of key parts of the pipeline including the modified QAOA ansatz and output post-processing. This data was collected in early 2024; here we compare our integrated solver’s hardware execution to the default execution achieved using Qiskit (version 0.44, with optimization level 3 and resilience level 1) and to random sampling. These data indicate that default circuit execution without additional error-suppression technology is indistinguishable from random sampling, and thus the quantum nature of the optimization plays no useful role. By contrast, the use of error-suppression leads to a separation of the distributions towards larger cut values, indicating more useful outputs from sampling the quantum circuit.

Next, we compare the quantum solver with and without post-processing to the classical local solver in order to disambiguate benefits coming from the quantum optimization and classical *post-processing*. In figures 4(d) and (e), we show the performance of the quantum and local solvers with and without post-processing for the spin-glass optimization problem presented in figure 3(c).

In both the quantum and classical cases, post-processing plays a significant role. The quantum solver exhibits a shift of its distribution to large cuts and achieves the correct Max-Cut solution with $\sim 8.5\%$ likelihood (with post-processing $AR \sim 82 \rightarrow 100\%$). The local solver (which includes postprocessing) also finds the correct solution after post-processing and sees a shift towards higher AR than simple random search, but with $\sim 500\times$ lower likelihood than the quantum solver. This is visible from the fact that only the right-hand tail of the local solver’s distribution overlaps the true maximum cut value, whereas the quantum solver’s distribution is skewed towards the correct answer. References [68, 69] show that the AR achieved through a greedy classical optimization algorithm alone for Max-Cut is expected to be 0.5. Nonetheless, as a post-processing step, it enables our solver to achieve much faster convergence to high-quality solutions and reduces the sensitivity of the algorithm to uncorrelated random bitflip errors. We also use this greedy algorithm as part of our θ update strategy.

Finally, we discuss the role of the modified variational ansatz and associated *parametric compiler* employed in the quantum solver. The ability to individually vary the angles θ allows the ansatz to begin in a nonuniform superposition that biases the QAOA circuit toward a low-cost solution, which we observe permits a reduction in the number of QAOA layers needed to converge on high-quality solutions. In our experiments, analysis of the convergence process indicates that the QAOA-variational-parameter optimization is necessary for convergence; i.e. we observe improvement from the quantum optimization beyond what is attainable in the $p = 0$ limit (see appendix C for further discussion).

Our approach adopts a parameterized initial state to reduce circuit depth necessary to achieve convergence. Nonetheless, even in the shallow limit, the number of two-qubit gates in each QAOA block increases dramatically with the connectivity and scale of the problem.

Implementing the modified QAOA ansatz requires efficient compilation to handle large-scale circuits. The Q-CTRL parametric compiler effectively reduces both circuit gate counts and duration compared to other commercially available parametric compilers [48, 49]. In appendix D, we compare the quality

of compilation, measured in terms of the number of two-qubit entangling gates required for key sample problems. These include $p = 2$ QAOA circuits of n -node Max-Cut problems over random 3-regular graphs and a more generic parametric circuit structure [70, 71]. Such structures are likely to appear for constrained optimization where the circuit ansatz may include non trivial mixing layers or more dense parameterization. The demonstrated compiler efficiency enables the greater performance of the optimization routine described above without incurring any substantive penalty in noise-susceptibility as might be expected by inclusion of the parameterized initial state to the ansatz.

5. Conclusion

In this manuscript we have introduced and benchmarked an end-to-end quantum solver for binary optimization problems that *successfully* solves NP-hard problems at the scale of 156 qubits. The methods we introduce consistently found the known optimal solution for Max-Cut and HUBO spin-glass problems; worst-case problem instances delivered ARs $\geq 99.5\%$. Moreover, when compared against the open literature for identical problem instances, these results demonstrate competitive or superior likelihood of sampling optimal solutions compared to published results using both trapped-ion gate-model machines and quantum annealers. Specifically, our solver finds the correct Max-Cut solution with up to $9\times$ higher likelihood than the best published trapped-ion results (which used classically pre-optimized circuit parameters from simulation) and over $1500\times$ higher likelihood than published quantum annealing results (which employed grid search over annealing parameters) for the spin-glass instances where direct comparison is possible, directly affecting the number of samples, and hence the execution time and cost, required to find a solution; these factors reflect differences in both the optimization methodology and hardware platform. While these comparisons involve different optimization methodologies (variational parameter optimization versus classically pre-optimized circuits versus grid search on annealers), they represent the most direct performance assessments possible given available published data. The new quantum solver was able to handle a broad range of problem classes and topologies, and relied exclusively on hardware execution without any classical simulation or pre-computation involved.

The results presented in this work demonstrate that an appropriately constructed quantum solver executed on an IBM gate-model quantum computer can outperform published outputs from quantum annealers for nontrivial binary optimization problems. Such a statement warrants careful qualification. First, we do not comment on the potential for different relative performance among gate-model machines should our quantum solver be implemented on alternative hardware. Next, as we relied upon published problem instances and implementations for quantum annealers we do not exclude the possibility of crafting better solutions in detailed implementation. We note that the higher-order binary optimization implemented by the authors of [36] was highly tailored to the problem and carefully constructed to efficiently use the available hardware resources in mapping a HUBO problem to the QUBO problem class efficiently addressed by annealers. Nonetheless it contains no optimality proof and an alternate implementation could deliver superior annealer results. Further, our results do not preclude the existence of alternate problem statements in which annealers retain advantages over gate-model machines, or that individual hardware systems may undergo periodic upgrades [72] to deliver improved performance relative to the reference results used here. Overall, despite these qualifications we believe these results now challenge the heretofore practically correct assertion that annealers would consistently outperform gate-model machines for quantum optimization problems.

These results also suggest the potential for significant advantages in future scaling of problem size and complexity leveraging the gate-model architecture. This is because the use of this architecture directly maintains the richness and flexibility to generalize to a broad class of problems beyond QUBO, as we show in the problems treated here. For the avoidance of any doubt we do not claim quantum advantage over or even practical equivalence with the highly efficient classical CPLEX solver. Nonetheless, the success of the quantum solver we have tested using all qubits available on the device provides a potential line of sight to solving much larger future problems which do challenge classical heuristics. Higher complexity problems lead to a higher gate count in the required circuits, but we believe increasing hardware scale and performance shows a potential path to quantum advantage in quantum optimization. We look forward to continued innovation in this space, including in novel alternate optimization algorithms that could be combined with our solver [73–76].

Comment on recent experimental demonstrations

We comment on recent experimental demonstrations published during preparation of this manuscript. First, [27] suggests signatures of quantum advantage on QAOA problems up to 109 qubits. The

problems treated were defined on a simpler linear topology and hardware implementations were not successful finding the correct solutions above 30 nodes. Further, the achieved ARs ($AR < 80\%$) fall below the mean AR values of 94%–97% attained by the purely classical local solver on our problem instances (table 2), suggesting that for problems of comparable or lesser difficulty, simple greedy heuristics may suffice to produce solutions of similar or superior quality. Results in [75] present an alternative method to QAOA, and deliver a roughly 30% enhancement over a standard implementation in simulation. Hardware execution of a pure QUBO problem at 100-qubit scale achieved $AR \sim 65\%$.

D-Wave provided a response to an earlier version of this manuscript in which they include an updated performance comparison to the Max-Cut and spin-glass results presented here [77]. We do not compare all of the results of our solver to QA as we did not have access to published data for all problem instances we treated, limiting our direct comparison to the category of high-order spin-glass problems. For such higher-order optimization problems, the D-Wave results published in [77] confirm that the integrated solver outperforms annealers for two of the cubic spin-glass problem instances, even with their new and augmented implementation that also includes our post-processing pass.

D-Wave also offers a comparison in which the team claims annealers maintain an advantage using their definition of a time-to-solution (TTS) metric. In this manuscript, we present only a comparison of probability for the final circuit from the QAOA process to sample the correct solution (comparable to execution of a D-Wave Hamiltonian following the embedding procedure), and we provide information on both sample time and runtime for the process (appendix A). We note an apparent discrepancy between D-Wave's definition of sample time and the actual sample time on hardware which leads to a difference of $\sim 100\times$ favoring annealers. The value of the sample time as defined therein is heavily dependent on the number of samples taken for the final circuit alone, and can be reduced by orders of magnitude by increasing this value in a way that actually increases total computation time. Without this $\sim 100\times$ factor, the TTS advantages they claim are neutralized or inverted in favor of gate-model implementations in the majority of cases treated. We therefore do not comment on the validity or utility of the TTS definition provided, or remark on the most appropriate performance metrics with respect to runtime. Further, in sparse Max-Cut problems, we do not claim or *a priori* expect our quantum solver to outperform QA. Prior to posting a draft of this manuscript no annealer data was available to enable a direct comparison; [77] introduces new results which surprisingly show the probability of finding the Max-Cut solution with our solver run on a gate-model machine is comparable to the new annealer implementation run by D-Wave. Thus, [77] appears to independently validate and strengthen the claims of our manuscript.

Acknowledgments

The authors are grateful to IBM Quantum for the provision of device access as well as access to the CPLEX classical optimization engine. We thank Andreas Bärtzsch, John Golden, Stephan Eidenbenz, Elijah Pelofske, and Reuben Tate for useful discussions and the sharing of data from their previous study. We also thank Ruslan Shaydulin for sharing data and IBM device access used during early development of this work. Finally, we are grateful to all other colleagues at Q-CTRL whose technical, product engineering, and design work has supported the results presented in this paper.

Data availability statement

The data cannot be made publicly available upon publication because they contain commercially sensitive information. The data that support the findings of this study are available upon reasonable request from the authors.

Appendix A. Resources and run time analysis

In previous sections, we analyzed the quality of the quantum solver compared to other solvers and previous demonstrations. Run time (cost) is another crucial metric for utility assessment. In this section, we provide details regarding the execution time and cost required for generating the data presented in this work. In table 1, we show results from Quantinuum and D-Wave as have appeared in previous publications. Using publicly available information, we approximate the runtime and cost required to produce the results shown.

Quantum solver (IBM): The optimization scheme we detail in the main text is designed for fast convergence. Our method batches six circuits at each optimization step in order to reduce overhead due to API and data transfer latency. The classical compute time between each two steps includes, for each of the six circuits, analysis of the data for cost extraction, the determination of the next set of circuit parameters, parameter binding and transpilation. In this work, the classical processing is performed sequentially for the six circuits and requires under 10 s, which totals to under two minutes throughout the full optimization loop. A straightforward parallelization of this step can effectively render the classical compute time negligible.

The smallest problems we solved (28–32 nodes Max-Cut) implemented 12 optimization steps, which totals to 72 circuits, each executed with 6000 shots for a total of 432 000 shots throughout the full optimization process. IBM's reported repetition time for this data is ~ 4000 shots per second, resulting in an ideal total QPU time of 108 s. In practice, the measured cumulative QPU time of these jobs is about 150 s. Additional API latency, overheads, and queue times on the hardware provider side exist and typically match the total QPU time (about 150 s), leading to a total wall-clock optimization time of about six minutes. Based on published IBM pricing, execution of the full optimization loop for these problems required approximately \$400 USD to run.

The larger problems we consider use 16–20 optimization steps, totaling 96–120 circuits, each with 8000–15 000 shots leading to a total of 768 000–1800 000 shots. A full optimization loop using the maximum number of shots required 7–8 min of QPU time and in the worst-case 20 min of total wall-clock time. Cost for such executions ranges $\sim \$1120$ – $\$1280$ USD.

Quantum annealers (D-Wave): Based on [37, 78], the dominant component of QPU time for QA is the anneal time which can be tuned between 0.5–2000 μs . In general, longer anneal times show better stability; in [78], the best anneal times were identified to be between 1000–2000 μs . On top of the anneal time, readout time of 250 μs and a programming time of 160 μs should be added for each sample.

The QA data in table 1, for three instances of the spin-glass problem, is taken from [36], where a total of 1328 000 samples were used for each problem. Assuming a sample time of 1000–1500 μs corresponds to 22–33 min of QA time.

The total optimization time includes the quantum execution time, along with the time for computing a minor embedding of the input to match the specific qubit connection structure inside the QPU, and the time to resolve solutions by mapping them back to the original (unembedded) problem. We have no information regarding the total execution time or cost for this system. [77] from D-Wave suggests a relevant sample time of 1.6 ms, making our estimate conservative, and provides general guidance on runtime of related problems.

Trapped-ion quantum computer (Quantinuum): [23], tested QAOA executions on Quantinuum H2-1 device. The results in table 1 represent a single circuit execution rather than a full iterative optimization process. We can estimate the cost of running such a circuit using available pricing information in [79]; we are not aware whether this published pricing is indicative of commercial terms available to Quantinuum customers. A 32-node 3-regular graph has 48 edges, hence, assuming arbitrary 2-qubit rotations are considered as basis gates and all to all connectivity, such a $p = 10$ QAOA circuit includes 480 2-qubit gates, 362 single-qubit gates and 64 reset/measurement operations (a requirement of higher-depth QAOA with $p > 1$ as implemented in [23] would require a proportionate growth in circuit resources). With 1024 shots, such a circuit consumes 1128 H-system quantum credits (HQC). With a standard subscription to H2 devices [79], one HQC is equivalent to \$13.5 USD, making the total cost of a single circuit execution $> \$15000$ USD.

We do not know whether full convergence of the optimization process on hardware would be possible as executed or whether if successful the convergence rate would differ from the implementation we describe above. However, assuming convergence equivalent to that achieved in the tests we have performed, full loop optimization over an equivalent 72 circuits would require \$1080 000 USD.

In [24, 25], the full optimization is done on the quantum computer. As reported, the optimization process for the 80-node Max-Cut problem lasted 11 h, with an achieved AR of 91%. Moreover, [24] mentions that a single sample (shot) time for a $p = 2$ QAOA of a 32-node 3-regular requires 0.97 sec while a $p = 1$ QAOA on the 130-node 3-regular problem (with qubit reuse on the 32-qubit device) requires 2.83 s. Using the information provided in [24], for the 32-node problem, 200 shots per circuit were used totaling approximately 90 min of QPU time. For larger and more complex problems, arriving at the optimal solution is likely to require higher-depth circuits and tens of thousands of shots.

Algorithm 1. GreedyPass.

Input: length- n bitstring $\mathbf{x}$, cost function C
Output: local minima bitstring

```

while  $\mathbf{x}$  not local minima of  $C$  do
  for  $j$  in  $\text{Shuffle}(n)$  do
     $\mathbf{x}' = \text{Flip}(\mathbf{x}, j)$ 
    if  $C(\mathbf{x}') < C(\mathbf{x})$  then
       $\mathbf{x} \leftarrow \mathbf{x}'$ 
    end if
  end for
end while
return:  $\mathbf{x}$ 

```

Algorithm 2. LocalSolver.

Input: sample size N , cost function C , passes K
Output: sample of local minima

Initialize $\mathbf{x}_i$ as N randomly sampled bitstrings

```

for  $i$  in  $1 : N$  do
   $c \leftarrow C(\mathbf{x}_i)$ 
   $\mathbf{x}_{\text{best}} \leftarrow \mathbf{x}'$ 
  for  $k = 1$  to  $K$  do
     $\mathbf{x}' \leftarrow \text{GreedyPass}(\mathbf{x}_i)$ 
    if  $C(\mathbf{x}') < c$  then
       $c \leftarrow C(\mathbf{x}')$ 
       $\mathbf{x}_{\text{best}} \leftarrow \mathbf{x}'$ 
    end if
  end for
   $\mathbf{x}_i \leftarrow \mathbf{x}_{\text{best}}$ 
end for
return:  $\{\mathbf{x}_i\}_{i=1, \dots, N}$ 

```

Appendix B. Post-processing and local solver

Classical post-processing: We use a naïve greedy optimization for the classical post-processing pass. Considering a bit flip to be a local move, bitstrings for which no local move improves the cost can be considered to be a local minimum of the problem. Greedy optimization corresponds to flipping single bits to improve the cost until a local minimum is reached, and no further improvement is possible. Pseudocode of the greedy optimization procedure is presented in algorithm 1. There, $\text{Shuffle}(N)$ randomly shuffles the bitstring indices $i = 0, \dots, n-1$, $\text{Flip}(\mathbf{x}, i)$ modifies bitstring $\mathbf{x}$ by flipping the i th bit. We limit the number of full traversals of the bitstring to a maximum of five passes.

The motivation for this pass is that the QAOA circuit produces a nontrivial bitstring distribution with support on bitstrings in the basin of good local minima. Device noise and measurement errors can be expected to perturb the bitstrings by local moves, but even a few erroneous bit flips can significantly decrease the solution quality. Greedy optimization corrects for this. The greedy post-processing scales as $\mathcal{O}(n)$ and therefore does not introduce any additional overhead.

Local solver: We implement a purely classical ‘local solver’ that samples bitstrings uniformly at random and applies a greedy post-processing pass to generate local minima of the cost function (see figure 4(e)). Here, a local minimum is defined to be a bitstring whose cost cannot be improved by local moves, in this case single bitflips. Pseudocode for the local solver is presented in algorithm 2. After first generating a set of random bitstrings, a greedy pass (algorithm 1) produces a local minimum by greedily flipping any bits that result in an improved cost (in algorithm 1, $\text{Flip}(\mathbf{x}, j)$ flips the j th bit of bitstring $\mathbf{x}$). Importantly, the order in which the local moves are considered is randomized (as indicated by the $\text{Shuffle}(n)$ term in the pseudocode). The greedy pass terminates when no cost-improving local moves exist. (In practice, the greedy pass terminates after a few bitflips, however, to avoid pathological edge cases, the pass is constrained through early stopping to no more than five full traversals of the bitstring.)

Since a given bitstring may be connected by cost-reducing local moves to multiple local minima, the local solver applies the greedy pass K times for each bitstring, each time using a different random seed. Since the order in which bits (spins) are traversed affects the outcome, each seed can lead to a different

local minimum. The best of the K minima produced is taken to be the post-processed bitstring. For this work, we set $K = 5$.

We note that such a local solver may be thought of as a zero-temperature version of simulated annealing (SA). In SA, similar sweeps are performed, however non-greedy moves are accepted with some probability that gradually approaches zero towards the end of the annealing process. Typical SA uses 500–5000 sweeps for each initial state (typically 100–1000 such states are considered).

Appendix C. Initial state parameterization

This appendix describes our modification to the standard QAOA initial state through parameterized biasing, the adaptive update rule that leverages intermediate results, and empirical assessment of how this enhancement improves convergence.

C.1. Initial state definition

The parameterized initial state construction corresponds to the layer of single-qubit R_y rotation gates shown in figure 2. This replaces the standard uniform superposition $|s\rangle = |+\rangle^{\otimes n}$ with a biased state that favors specific bitstrings while maintaining the ability to explore the solution space.

We can understand this construction by first examining the action of $R_y(\theta)$ on a single qubit $|0\rangle$, which produces:

$$R_y(\theta)|0\rangle = \cos\left(\frac{\theta}{2}\right)|0\rangle + \sin\left(\frac{\theta}{2}\right)|1\rangle. \quad (5)$$

The standard QAOA initial state is recovered by setting $\theta = \pi/2$ to create an equal superposition. To introduce bias toward a target bit value $b_T \in \{0, 1\}$, we instead parameterize the rotation angle as

$$\theta = \frac{\pi}{2} - (-1)^{b_T} \delta, \quad (6)$$

where $\delta \in [0, \pi/2]$ serves as the biasing parameter. This yields a measurement probability $\Pr(b_T) = (1 + \sin \delta)/2$ for the target bit. The parameter δ controls the bias strength: $\delta = 0$ recovers the uniform superposition, while $\delta = \pi/2$ produces a deterministic state $|b_T\rangle$.

For the n -qubit case, given a target length- n bitstring $\mathbf{b}_T \in \{0, 1\}^n$, we construct the biased initial state as:

$$|\psi_\delta(\mathbf{b}_T)\rangle = \bigotimes_{i=1}^n R_y\left(\frac{\pi}{2} - (-1)^{(b_T)_i} \delta\right) |0\rangle_i = \sum \mathbf{c}_b |\mathbf{b}\rangle. \quad (7)$$

Since the state is a product state, the probability of sampling any bitstring $\mathbf{b}$ factorizes into independent per-qubit contributions, with each qubit biased toward the corresponding target bit value by an amount controlled by δ . This product state has a particularly useful property: the squared amplitude of any computational state $|\mathbf{b}\rangle$ depends only on the biasing angle δ and the Hamming distance $d_H(\mathbf{b}, \mathbf{b}_T)$ and is given by

$$|c_b|^2 = \left(\frac{1 + \sin \delta}{2}\right)^n \left(\frac{1 - \sin \delta}{1 + \sin \delta}\right)^{d_H(\mathbf{b}, \mathbf{b}_T)}. \quad (8)$$

Note that when $\delta = 0$, the unbiased case, $|c_b|^2 = 2^{-n}$ for all $\mathbf{b}$.

Equation (7) defines a single-parameter family of product states where measurement probabilities from this state decrease monotonically with Hamming distance from the target bitstring.

C.2. Measurement-feedback-based initial state biasing

When executing the solver on hardware, each circuit is sampled a number of times (according to the shot count) as part of the closed-loop variational parameter (β, γ) optimization procedure. Each shot results in a bitstring with an associated cost, $C(\mathbf{x})$. Our algorithm leverages the intermediate bitstring cost information by introducing a multi-stage optimization procedure. Each stage consists of multiple optimization steps where the β, γ variational parameters are optimized, but the initial state parameters remain fixed. Between each stage, the initial state parameters are updated.

The initial state replaces the traditional Hadamard gates of standard QAOA with parameterized single-qubit R_y gates (see equation (7)), with the angle defined as shown in equation (6). For the first

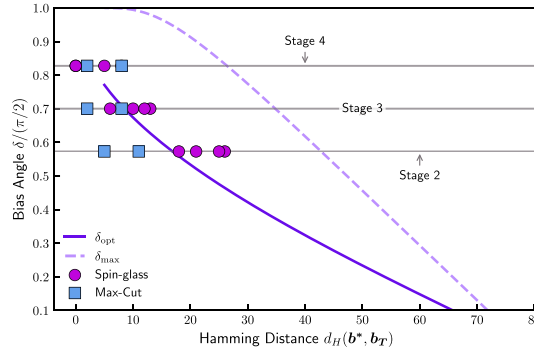


Figure 5. Theoretical bounds for optimal and maximum bias angles for $n = 156$. The gray horizontal lines represent the bias angle δ schedule ($\delta = [0.0, 0.9, 1.1, 1.3]$ rad) used for all data presented in this work irrespective of problem size. The first stage in the multi-stage optimization always has $\delta = 0$ recovering the standard QAOA ansatz. For this illustration, we include the $n = 156$ data, which includes two Max-Cut instances and four cubic spin-glass instances. The data points represent the hamming distance between the optimal solution returned at the end of the stage and the target bitstring used for biasing the initial state for all circuits during that stage.

stage, δ , the biasing strength parameter, was set to 0, recovering the standard QAOA ansatz. In subsequent stages, increasing values of δ were set from a predetermined schedule, using the lowest-cost bitstring observed thus far as the target $\mathbf{b}_T$. δ were chosen such that the algorithm progressively localizes the initial state superposition around promising solutions while maintaining sufficient exploration through the QAOA dynamics. Our current δ schedule was determined empirically to work well for a range of problems and problem sizes; therefore, we chose to use the same schedule for all runs presented in this work to $\delta = [0.0, 0.9, 1.1, 1.3]$ rad. In this section, we use the 156-qubit data collected to illustrate how the schedule chosen compares to theoretical limits for $n = 156$.

Biasing can negatively impact our initial state if it is too strongly biased toward a bitstring $\mathbf{b}_T$ (assuming $\mathbf{b}_T$ is not an optimal solution to the problem, $\mathbf{b}^*$). If the bias angle, δ , is sufficiently large, the squared amplitude of $|\mathbf{b}^*\rangle$, can be less than the squared amplitude when no bias is applied. That is, for some $d_H(\mathbf{b}^*, \mathbf{b}_T)$, there is a range $\delta \in [0, \delta_{\max}]$ such that $|c_{b^*}|^2 > 2^{-n}$, and outside of this range is less than 2^{-n} .

We can determine two important bias angles for a particular problem size from equation (8): an optimal bias angle, δ_{opt} , and a maximum bias angle, $\delta_{\max}$. The optimal bias angle is found maximizing equation (8), given by $\delta_{\text{opt}} = \sin^{-1}((n - 2h)/n)$, where $h = d_H(\mathbf{b}, \mathbf{b}^*)$. The maximum bias angle is set by finding the angle such that $|c_{b^*}|^2 = 2^{-n}$, which is equivalent to finding the non-zero angle that satisfies $(n - h) \ln(1 + \sin \delta) + h \ln(1 - \sin \delta) = 0$.

In figure 5, we illustrate $\delta_{\max}$ and δ_{opt} for $n = 156$. We also show the bias schedule used for all data presented in this work: $\delta = [0.0, 0.9, 1.1, 1.3]$ rad for stages $s = 1, \dots, 4$, respectively. This single schedule was used across all problem sizes and instances without modification.

The complete algorithm is presented in algorithm 3, which takes as input the cost Hamiltonian H_C , the number of stages S , the number of iterations per stage T , and the biasing schedule $\{\delta_s\}_{s=1}^S$. Within each stage, the initial state parameterization remains fixed while the QAOA parameters are optimized using CMA-ES with CVaR as the objective function. Between stages, the initial state is updated to bias toward the best solution found, enabling efficient exploration of progressively refined search regions.

C.3. Empirical evaluation

To illustrate the impact of our biased initial state parameterization on algorithm performance, we examine the optimization performance for a representative spin-glass optimization run. Figure 6 presents a complete optimization run for spin-glass problem instance 5, tracking the best CVaR cost evolution across all stages and iterations as well as showing the CVaR values and variational parameters for all circuits run during the optimization process.

For each of the 16 optimization steps, split into four stages, we submit six circuits to the quantum computer. Each circuit submitted within each stage has the same initial state θ values, optimizing only the γ and β parameters in the CMA-ES loop. For the first step of each stage, we include one circuit where $\beta = \gamma = 0$, which is the equivalent of a $p = 0$ circuit only containing the initial state followed by measurement, and for the other five circuits, the variational parameters are chosen randomly. The CMA-ES optimizer ($\sigma = 0.1$) then uses the CVaR cost ($\alpha = 0.35$) of each measured distribution to define the variational parameters for the next iteration. The choice of $\alpha = 0.35$ is a conservative value below the

Algorithm 3. BiasedQAOA.**Input:** Cost function C , stages S , iterations per stage T , bias schedule $\{\delta_s\}_{s=1}^S$ **Output:** Optimized bitstring $\mathbf{x}_{\text{best}}$ Initialize $\boldsymbol{\theta} \leftarrow \pi/2 \cdot \mathbf{1}_n$ $\triangleright$ Uniform superpositionInitialize $\mathbf{x}_{\text{best}} \leftarrow \text{null}$ **for** stage $s = 1$ to S **do** **if** $s > 1$ **then** Update $\boldsymbol{\theta}$ using equation (7) with $\mathbf{b}_T = \mathbf{x}_{\text{best}}$ and $\delta = \delta_s$ **end if** **for** iteration $t = 1$ to T **do**

Execute circuits and measure bitstrings

 Optimize $\boldsymbol{\gamma}, \boldsymbol{\beta}$ using CMA-ES with CVaR objective **end for**

Apply GreedyPass to best-circuit samples

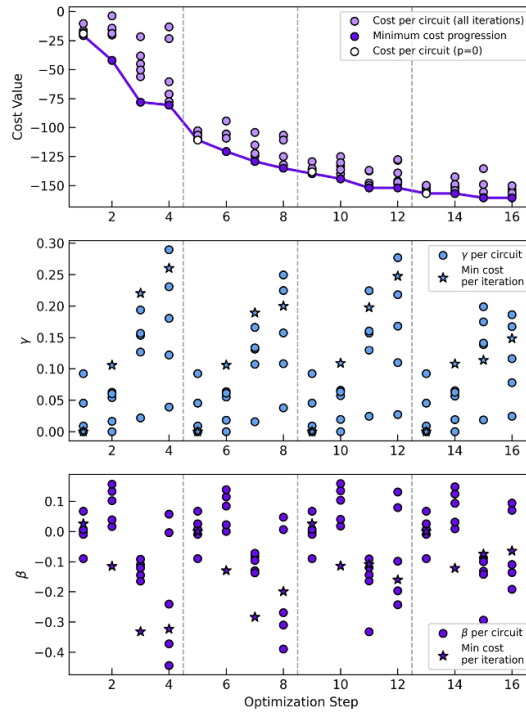
 Update $\mathbf{x}_{\text{best}}$ if improved**end for****return:** $\mathbf{x}_{\text{best}}$ 

Figure 6. Top panel: CVaR cost ($\alpha = 0.35$) per solver iteration for 127-qubit spin-glass problem instance 5 (the full output distribution for this instance appears in figure 10(c)). The dark purple data points are the minimum cost observed up to and including that optimization step, and the light purple points are the CVaR cost evaluated for each circuit during the optimization process. There are six circuit evaluations per step. The vertical lines in each panel represent each time the $\boldsymbol{\theta}$ angles were updated during the optimization. In the first step and also in the first step after each vertical line (optimization steps 1, 5, 9, and 13) we include one circuit with variational parameters $(\boldsymbol{\gamma}, \boldsymbol{\beta}) = (0, 0)$ as a baseline for comparison (highlighted in white). The CVaR cost, which is used during the hybrid optimization loop, is evaluated for each circuit after measurement error mitigation [40] prior to the bitflip post-processing. Lower panels: The values of the QAOA parameters $\boldsymbol{\gamma}$ (middle) and $\boldsymbol{\beta}$ (bottom) during the optimization (we use $\sigma = 0.1$ for CMA-ES and $\boldsymbol{\gamma}$ is restricted to positive values). While the initial QAOA parameters are generated randomly at the beginning of the optimization run, we chose to use the same random seed when generating points after the initial state updates in order to isolate and observe the role of the initial state only.

$\alpha < 0.5$ regime where [53] reports the strongest convergence improvements; it balances convergence speed (favored by smaller α) against robustness to sampling noise at finite shot counts. This value was fixed across all problem instances without per-instance tuning.

The $\boldsymbol{\theta}$ angles are all initialized to $\pi/2$ and the initial state is unchanged until the second stage. Therefore, the first four steps of the optimization are equivalent to a standard QAOA execution.

After four steps, the $\boldsymbol{\theta}$ angles are updated and a new stage begins, indicated by the vertical dashed lines in the figure. When the initial state parameters are updated, the following optimization step again

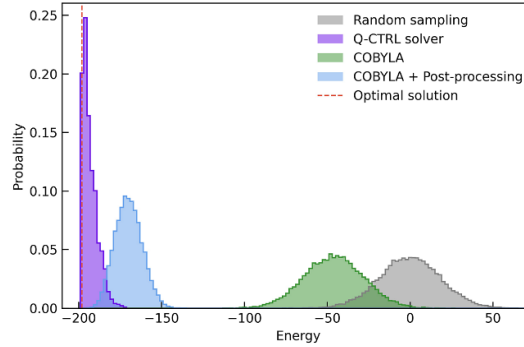


Figure 7. Initial state optimization using COBYLA. For this simulation, we use the same parameterized initial state as our modified QAOA ansatz, but remove all QAOA layers ($p = 0$). We use COBYLA to optimize all R_y angles directly after initializing them to $\pi/2$ using the cost function from 127-qubit spin-glass instance 5 and CVaR with $\alpha = 0.35$ (the experimental results for this instance appear in figure 10(c) and are reproduced here). We run for a maximum of 100 iterations after which the optimization is stopped. Following the optimization, we apply our postprocessing pass described in appendix B to the optimized distribution.

includes one circuit for which the QAOA parameters $\beta = \gamma = 0$, equivalent to a $p = 0$ version of the algorithm with the modified initial state ansatz. Accordingly, steps 1, 5, 9, and 13 in figure 6 each include one $p = 0$ circuit as one of the six data points used by the CMA-ES optimizer in optimization of the circuit parameters and are highlighted in figure 6 with white circle markers. The initial state is unchanged within each stage; i.e. the circuits submitted to hardware in steps 1–4, 5–8, 9–12, and 13–16 have identical initial states, just different values for β, γ . Each time the initial state is updated, we again randomly choose starting values for β, γ , but we chose to use the same random seed in order to isolate the role of the initial state.

We update the initial state by using the best solution found across previous iterations to define a new nonuniform starting superposition that biases the initial state towards the target bitstring. The biasing angle δ increases for each stage. As shown in figure 6, we observe that every time the initial state is updated, there is an improvement in the CVaR cost. This may be attributed to the fact that increasing the biasing angle narrows the support of the initial state around the best-observed computational state. We also observe an improvement in the cost when the initial state is unchanged, and only the QAOA parameters are changed. This observation indicates that the QAOA circuit parameter optimization plays a role in solving the problem, beyond the initial-state parameterization.

In the same plot, we also show explicitly the values of β, γ throughout the optimization. Each time the initial state is changed, the β, γ angles are initialized to the same randomly generated points from the beginning of the optimization in order to isolate the performance from the initial state parameterization. We only perform this optimization for four steps before updating the initial state in order to reduce as much as possible the resources required for convergence. In figure 6, we see on average the β, γ parameters increase in magnitude when optimized.

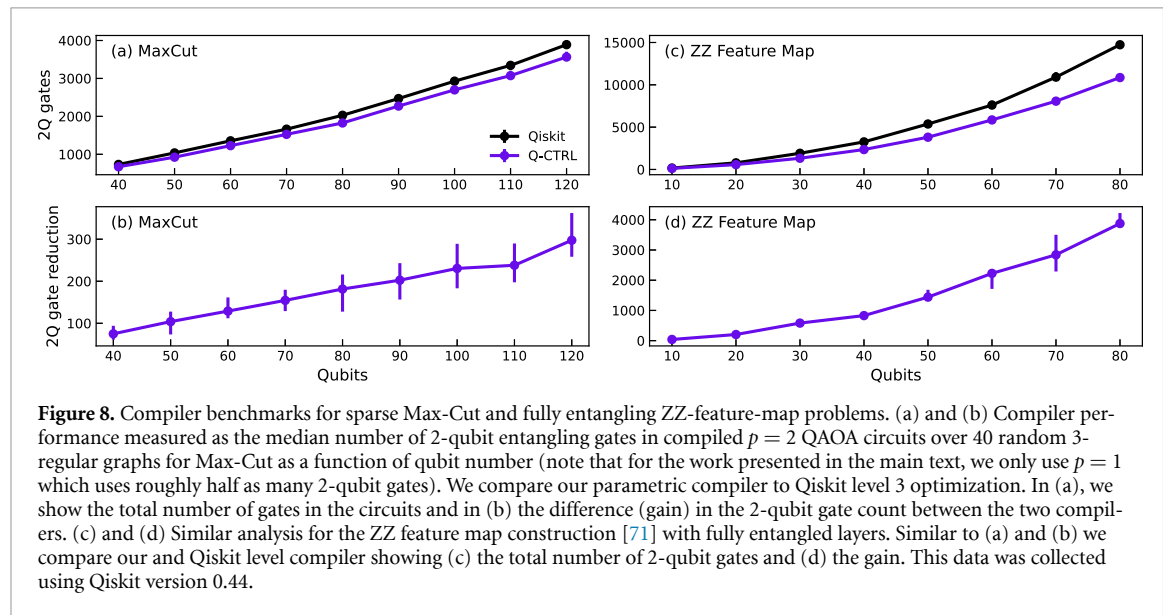
Lastly, we consider the possibility that the initial state parameterization alone may be responsible for the observed performance of our solver. To exclude this hypothesis, we consider a noiseless simulation of a $p = 0$ circuit corresponding a simple product state with no entanglement. The circuit is taken to be simply a product of R_y gates, the same as used in our initial state, but without the restriction used in the biasing, equation (6). Instead, the $\theta \in \mathbb{R}^n$ angles are allowed to take on arbitrary values. These are optimized using a number of objective function evaluations chosen to match the number of circuit executions as a typical run (~ 100). Here we use COBYLA rather than the CMA-ES optimizer used in the rest of this work, as the CMA-ES default population size scales with the number of parameters [80], making it impractical for optimizing n angles (e.g. 127 or 156) within the available budget of ~ 100 circuit executions chosen to match the total number of circuits submitted on average during our hardware runs. With a reasonable population size, CMA-ES would be limited to approximately 6 optimization steps, which is insufficient for convergence. Figure 7 shows that the resulting optimized product state, with or without the greedy post-processing, is unable to match the results of our algorithm. This constitutes numerical evidence against the hypothesis that the observed performance of our solver can be attributed to the initial state parameterization alone and confirms that QAOA entanglement dynamics contribute to the observed results.

Appendix D. Parametric compilation analysis

The Q-CTRL *parametric compiler* effectively reduces both circuit gate counts and duration compared to other commercially available parametric compilers. Here, we compare the quality of compilation, measured in terms of the number of two-qubit entangling gates required for circuit execution. We treat two extremal problems which bound the expected envelope of circuit structures to be encountered in generic optimization problems.

In figures 8(a) and (b) show $p = 2$ QAOA circuits of n -node Max-Cut problems over 40 different random 3-regular graphs, similar to the problems we show in the main text. We observed a $\sim 10\%$ reduction in the number of 2-qubit gates for each circuit which amounts to a reduction of ~ 300 gates at the maximum tested 120 nodes (qubits).

In figures 8(c) and (d) we consider a more generic parametric circuit structure [70, 71]. In this example, we use circuits with fully entangling layers, resulting in deep circuits. Such structures are likely to appear for constrained optimization where the circuit ansatz may include non trivial mixing layers or more dense parameterization. We observed $> 25\%$ reduction in the number of 2-qubit gates which amounts to a reduction of ~ 4000 gates for 80-qubit circuits.



Appendix E. Additional data

In this section, we provide additional data about the different problem instance tested. Comprehensive information about each problem execution is presented in table 2.

There we provide for each problem instance the number of optimization steps used (each optimization step contains six circuits), and the number of shots (samples) used for each circuit. Each quantum circuit execution during the optimization process uses the same number of shots. The information regarding best solution found and likelihood always refer to the optimal circuit (minimum cost) only. Additional information on each problem instance includes: the number of times the top solution was sampled, the number of unique optimal configurations found, the mean value of the objective function across all samples and the approximation ratio (AR) of the mean.

In the main text, we present the AR with respect to the best solution identified by the solver. Another common convention evaluates the AR using the mean value over all samples. In particular, this convention is informative for small problems where the number of samples—either of the optimal circuit or throughout the full optimization process—is not negligible compared to the number of possible configurations. None of problems we consider have this property; we provide here additional information about the mean values for completeness.

Table 2. Additional information to table 1 in the main text regarding the Q-CTRL solver and the Local solver. The set of problems we show here is identical. Both the ideal Min/Max values and the ones found by each solver are shown in the main text and are presented here for completeness. *Shots* represents the number times the optimal circuit, found by the quantum solver, was sampled and the number of initial configurations used by the local solver, *Mean* represents the averaged objective value (max cut or min energy) across all samples (shots), *Mean AR* is the approximation ratio of the mean value, *Count* is the number of times the top solution, found by each solver, was sampled, *Unique* is the number of unique optimal configurations each solver found and *Total shots* is the total amount of shots used by the quantum solver throughout the full optimization process.

Max-Cut

Instance	Global		Q-CTRL (IBM Quantum) Full hybrid optimization						Local solver				
	Min/Max	Shots	Max	Mean	Mean AR	Count	Unique	Total shots	Max	Mean	Mean AR	Count	Unique
(28, 3, 102u)	0/40	6k	40	39.82	0.995	5661	2	432k	40	38.82	0.971	3718	4
(30, 3, 264u)	0/43	6k	43	42.75	0.994	5511	1	432k	43	41.43	0.963	2810	2
(32, 3, 7u)	0/46	6k	46	45.42	0.987	4949	1	432k	46	43.93	0.955	2433	2
(80, 3, 68u)	0/106	10k	106	104.23	0.983	1383	1	1.20 M	106	101.79	0.960	1	1
(100, 3, 12u)	0/135	10k	135	132.18	0.979	1214	2	1.20 M	135	128.86	0.955	1	1
(120, 3, 8u)	0/163	12k	163	159.55	0.979	1031	10	1.15 M	163	155.04	0.951	2	2
(156, 3, 0u)	0/213	15k	213	210.31	0.987	3081	30	1.62 M	211	201.08	0.926	2	2
(156, 5, 0u)	0/324	15k	324	311.95	0.963	32	6	1.53 M	322	309.07	0.954	1	1
(50, 6, 28w)	0/73.20	10k	73.20	71.43	0.976	810	4	960k	73.20	71.11	0.971	367	4
(50, 7, 27w)	0/86.25	10k	86.25	84.39	0.978	771	2	960k	86.25	84.04	0.974	367	2
(60, 5, 36w)	0/76.25	10k	76.25	74.01	0.971	1013	5	960k	76.25	73.22	0.960	157	6
(70, 4, 75w)	0/83.75	10k	83.75	80.58	0.962	616	6	960k	83.75	78.96	0.943	22	11
(80, 4, 46w)	0/88.00	10k	88.00	84.01	0.955	208	12	960k	88.00	83.23	0.946	29	12

Spin glass

Instance	Global		Q-CTRL (IBM Quantum) Full hybrid optimization						Local solver				
	Min/Max	Shots	Min	Mean	Mean AR	Count	Unique	Total shots	Min	Mean	Mean AR	Count	Unique
0	−200/192	15k	−200	−188.43	0.970	2	1	1.44 M	−188	−149.98	0.872	1	1
3	−198/184	15k	−198	−190.37	0.980	1768	33	1.44 M	−190	−152.63	0.881	1	1
5	−198/192	15k	−198	−193.15	0.988	3045	129	1.44 M	−188	−151.75	0.881	4	4
10	−202/190	15k	−202	−193.42	0.978	288	7	1.44 M	−194	−146.14	0.858	1	1
11	−180/196	15k	−178	−173.25	0.982	2530	86	1.44 M	−170	−142.98	0.902	1	1
69	−190/212	15k	−188	−182.97	0.983	2630	27	1.80 M	−180	−148.52	0.897	1	1
0 (156Q-K)	−242/242	15k	−242	−236.99	0.990	3254	471	1.80 M	−226	−184.63	0.882	2	2
0 (156Q-B)	−242/242	15k	−242	−238.14	0.992	2	2	1.80 M	−226	−184.63	0.882	2	2
1 (156Q-B)	−240/238	15k	−240	−238.08	0.996	6964	1009	1.80 M	−228	−183.65	0.882	1	1
2 (156Q-B)	−250/246	15k	−250	−246.43	0.993	25	10	1.80 M	−230	−185.63	0.870	1	1

In figures 9 and 10, we present further information for selected problem instance from table 1. Figure 9 six selected Max-Cut instances and figure 10 shows the six 127-qubit spin-glass problem instances studied. Figure 11 shows all 156-qubit problems studied (both Max-Cut and cubic spin-glass instances).

In all, the left column shows the objective function (ground state energy or maximum cut value) distribution for N (given for each problem in table 2) sampled configurations produced by three different approaches: sampling configurations uniformly at random (Random sampling), a local solver that applies the greedy optimization of algorithm 1 to randomly-sampled configurations (Local solver), and the Q-CTRL solver. The true ground-state energy (red dashed line) is calculated using CPLEX. The right column shows the cumulative distribution function (CDF) of the AR for the same data as in the adjacent panel. The vertical dashed lines indicate the maximal AR found by each solver. The x -axis value indicates which fraction of the sampled configurations (out of N) has an AR equal or larger than the value on the y -axis.

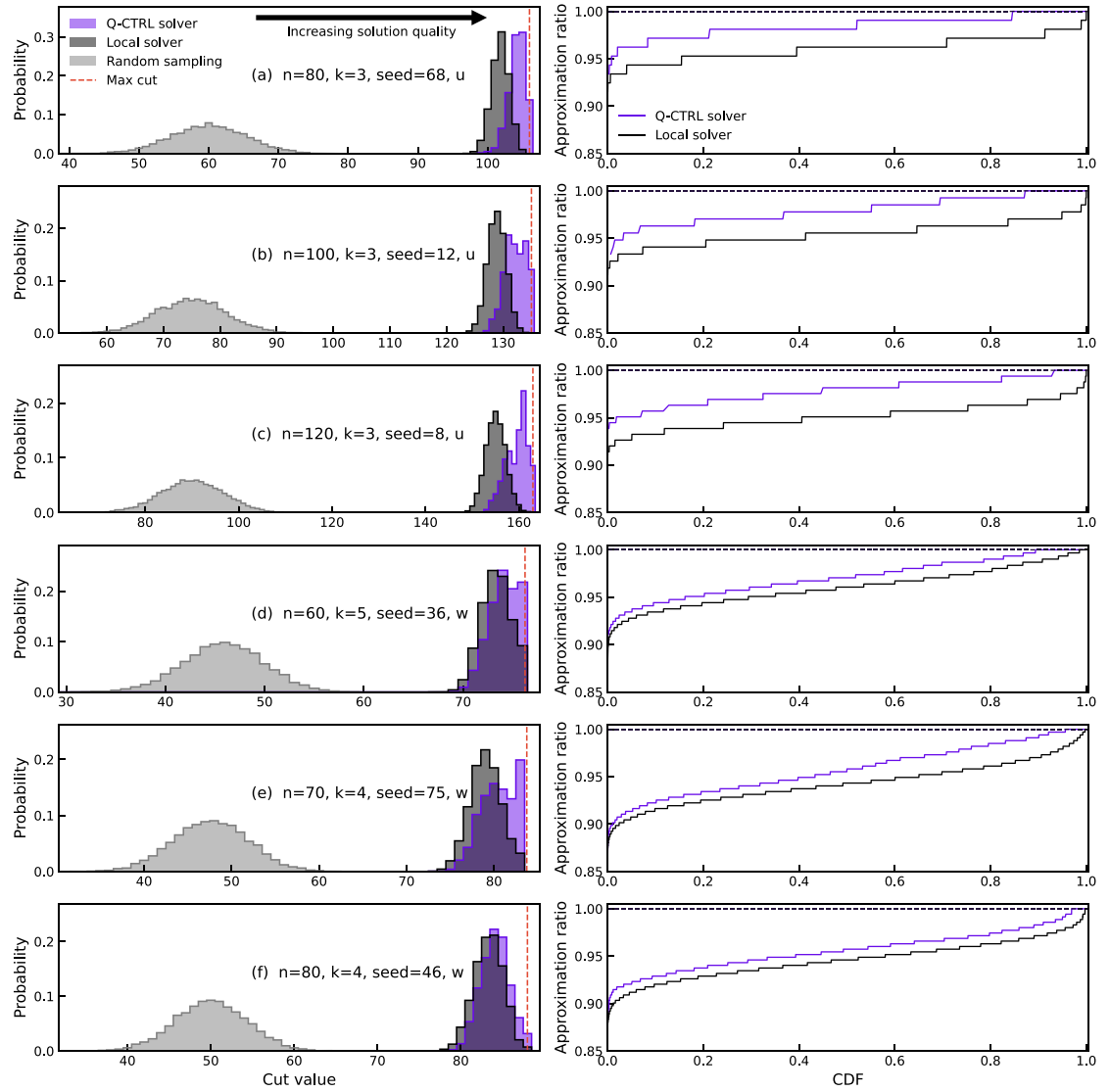


Figure 9. Distribution of optimized-circuit outcomes for various Max-Cut problems. (Left column) The cut value distribution for $N = 6\text{--}12\text{k}$ sampled configurations (see the ‘Shots’ column in table 2) produced by three different approaches: sampling configurations uniformly at random (Random sampling), a local solver that applies the greedy optimization of algorithm 1 to randomly-sampled configurations (Local solver), and the solver presented in this work. The true maximum cut value (red dashed line) is calculated using CPLEX. (Right column) The cumulative distribution function (CDF) of the approximation ratio for the same data as in the adjacent panel. The vertical dashed lines indicate the maximal AR found by each solver (the correct solution corresponds to AR = 1). See tables 1 and 2 for full statistics for these instances.

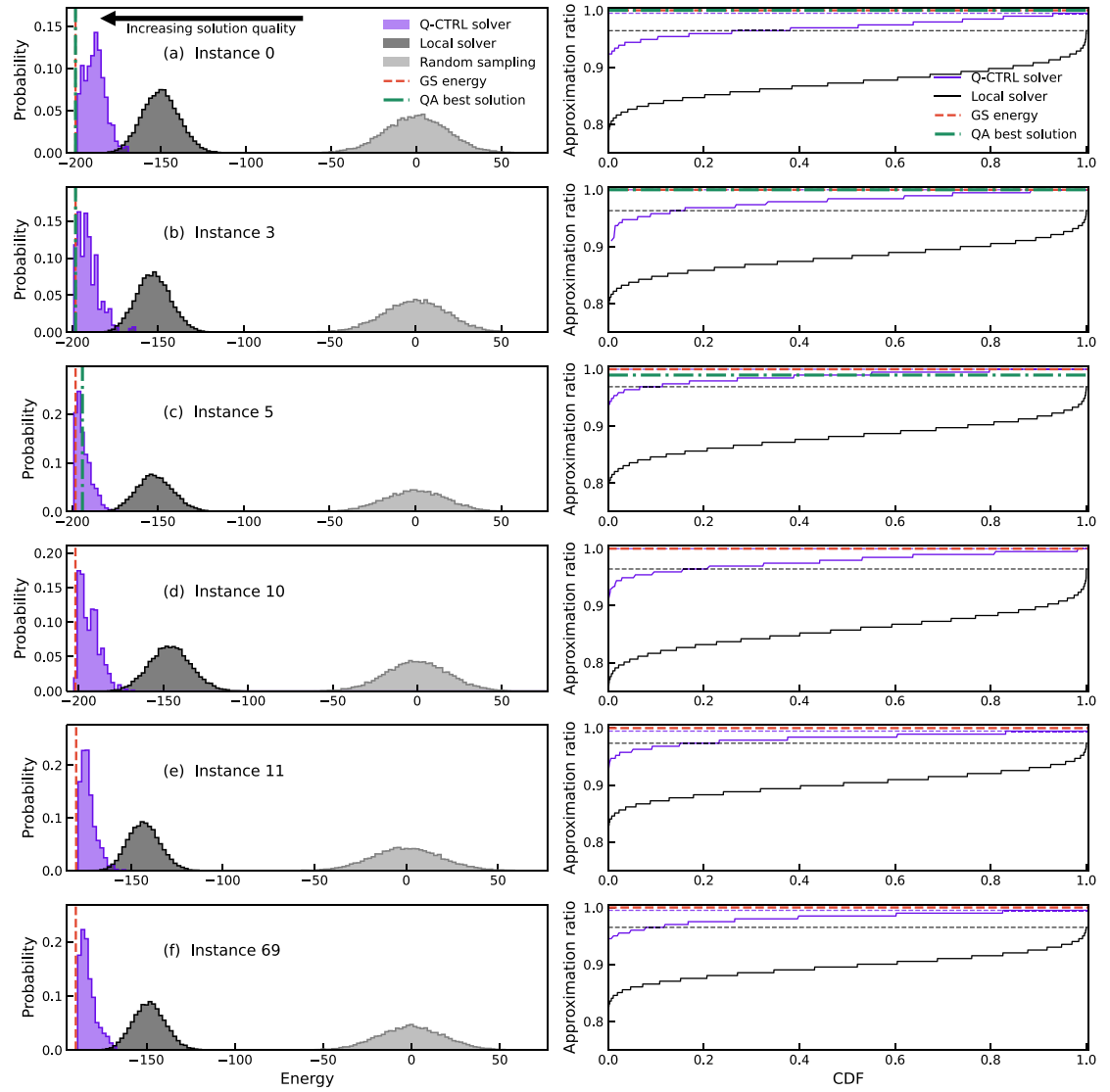


Figure 10. Distribution of optimized-circuit outcomes for the spin-glass problem instances on 127 qubit IBM devices. (Left column) The energy distribution for 15 000 sampled configurations produced by three different approaches: sampling configurations uniformly at random (Random sampling), a local solver that applies the greedy optimization of algorithm 1 to randomly-sampled configurations (Local solver), and the Q-CTRL solver. The true ground-state energy (red dashed line) is calculated using CPLEX. (Right column) The cumulative distribution function (CDF) of the approximation ratio (AR) for the same data as in the adjacent panel. The horizontal dashed lines indicate the maximal AR found by each solver (the correct solution corresponds to $AR = 1$). See tables 1 and 2 for full statistics for these instances.

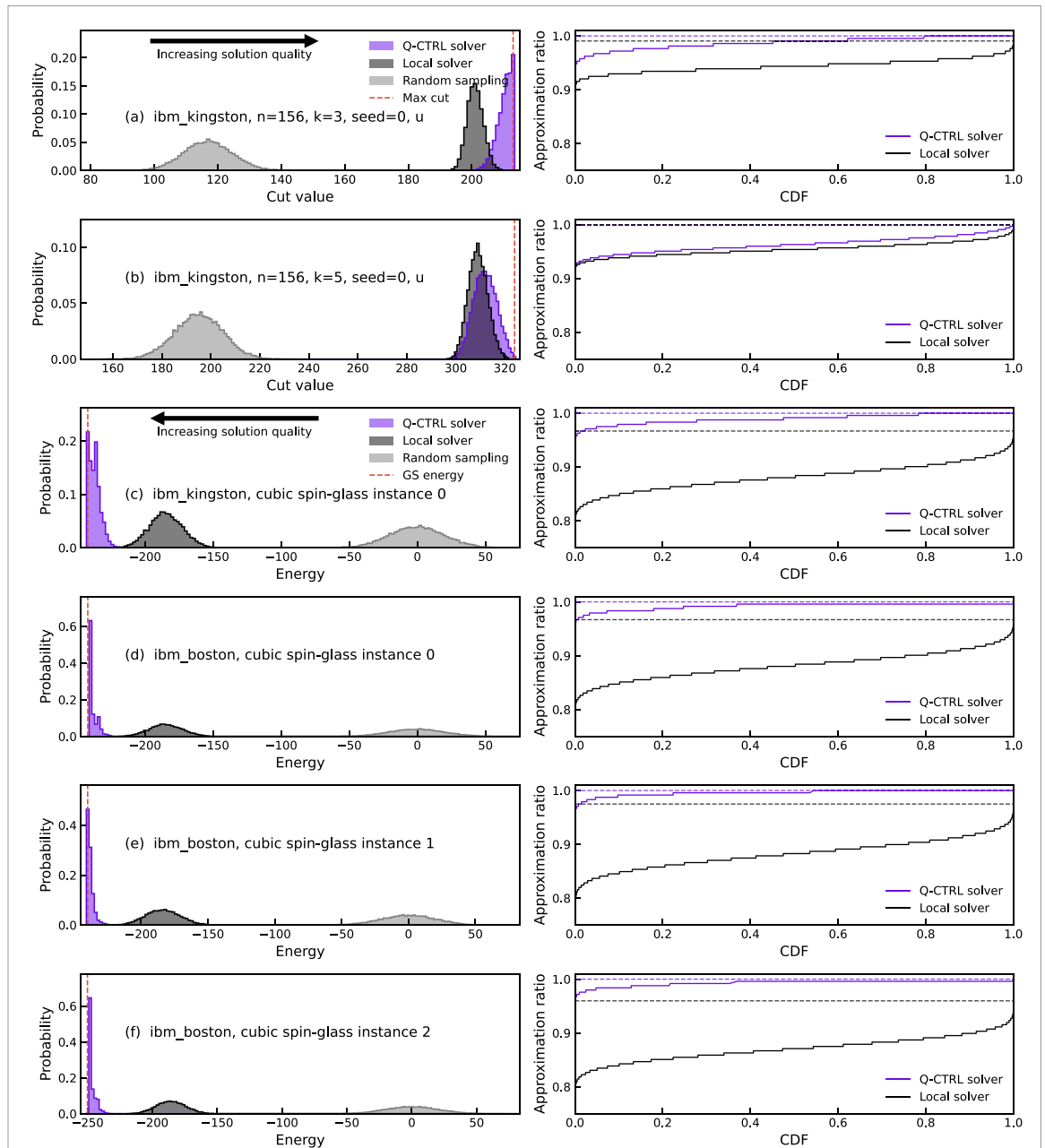


Figure 11. Distribution of optimized-circuit outcomes for the Max-Cut and spin-glass problem instances on 156 qubit devices. (Left column) The energy distribution for 15 000 sampled configurations produced by three different approaches: sampling configurations uniformly at random (Random sampling), a local solver that applies the greedy optimization of algorithm 1 to randomly-sampled configurations (Local solver), and the Q-CTRL solver. The optimal cut value for Max-Cut and true ground-state energy for the spin-glass (red dashed line) is calculated using CPLEX. The green dashed line represents the best result using quantum annealing for those instances for which annealer data is available. (Right column) The cumulative distribution function (CDF) of the approximation ratio (AR) for the same data as in the adjacent panel. The horizontal dashed lines indicate the maximal AR found by each solver (the correct solution corresponds to $AR = 1$). See tables 1 and 2 for full statistics for these instances.

ORCID iDs

Natasha Sachdeva 0000-0001-9305-4208
 Gavin S Hartnett 0000-0002-6814-1809
 Smarak Maity 0000-0002-8431-7480
 Yulun Wang 0000-0002-0847-0248
 Ryan Dougherty 0009-0003-4795-9104
 G Adam Cox 0009-0005-9502-9504
 Michael Hush 0000-0002-6464-9234
 Pranav S Mundada 0000-0003-3316-0724
 Michael J Biercuk 0000-0002-4371-2580
 Yuval Baum 0000-0003-4631-8551

References

- [1] Gao X, Du H and Han M 2017 *Combinatorial Optimization and Applications: 11th Int. Conf., COCOA 2017, (Shanghai, China, 16 December–18 December 2017) (Proc., Part I)* (Springer)
- [2] Preskill J 2018 Quantum computing in the NISQ era and beyond *Quantum* **2** 79
- [3] IBM Corporation 2024 IBM ILOG CPLEX Optimization Studio
- [4] Gurobi Optimization, LLC 2023 Gurobi Optimizer Reference Manual
- [5] Rehfeldt D, Koch T and Shinano Y 2023 Faster exact solution of sparse maxcut and QUBO problems *Math. Program. Comput.* **15** 445–70
- [6] Borndörfer R, Lindner N and Roth S 2020 A concurrent approach to the periodic event scheduling problem *J. Rail Transp. Plan. Manage.* **15** 100175
- [7] Vidal T, Crainic T G, Gendreau M and Prins C 2014 A unified solution framework for multi-attribute vehicle routing problems *Eur. J. Oper. Res.* **234** 658–73
- [8] Boulebane S and Montanaro A 2024 Solving boolean satisfiability problems with the quantum approximate optimization algorithm *PRX Quantum* **05** 030348
- [9] Guerreschi G G and Matsuura A Y 2019 QAOA for max-cut requires hundreds of qubits for quantum speed-up *Sci. Rep.* **9** 6903
- [10] Shaydulin R et al 2024 Evidence of scaling advantage for the quantum approximate optimization algorithm on a classically intractable problem *Sci. Adv.* **10** eadm6761
- [11] Blekos K, Brand D, Ceschini A, Chou C-H, Li R-H, Pandya K and Summer A 2024 A review on quantum approximate optimization algorithm and its variants *Phys. Rep.* **1068** 1–66
- [12] Yarkoni S, Raponi E, Bäck T and Schmitt S 2022 Quantum annealing for industry applications: introduction and review *Rep. Prog. Phys.* **85** 104001
- [13] Deng Z, Wang X and Dong B 2023 Quantum computing for future real-time building HVAC controls *Appl. Energy* **334** 120621
- [14] Verchère Z, Elloumi S and Simonetto A 2023 Optimizing variational circuits for higher-order binary optimization *IEEE Int. Conf. on Quantum Computing and Engineering (QCE)* (<https://doi.org/10.1109/QCE57702.2023.00011>)
- [15] Glos A, Krawiec A and Zimborás Z 2022 Space-efficient binary optimization for variational quantum computing *npj Quantum Inf.* **8** 39
- [16] Niroula P, Shaydulin R, Yalovetzky R, Minssen P, Herman D, Hu S and Pistoia M 2022 Constrained quantum optimization for extractive summarization on a trapped-ion quantum computer *Sci. Rep.* **12** 17171
- [17] Fuchs F G, Lye K O, Møll Nilsen H, Stasik A J and Sartor G 2022 Constraint preserving mixers for the quantum approximate optimization algorithm *Algorithms* **15** 202
- [18] Farhi E, Goldstone J and Gutmann S 2014 A quantum approximate optimization algorithm (arXiv:1411.4028)
- [19] Niu S and Todri-Sanial A 2022 Effects of dynamical decoupling and pulse-level optimizations on IBM quantum computers *IEEE Trans. Quantum Eng.* **3** 1–10
- [20] Weidenfeller J, Valor L C, Gacon J, Tornow C, Bello L, Woerner S and Egger D J 2022 Scaling of the quantum approximate optimization algorithm on superconducting qubit based hardware *Quantum* **6** 870
- [21] Harrigan M P et al 2021 Quantum approximate optimization of non-planar graph problems on a planar superconducting processor *Nat. Phys.* **17** 332–6
- [22] Sack S H and Egger D J 2024 Large-scale quantum approximate optimization on nonplanar graphs with machine learning noise mitigation *Phys. Rev. Res.* **6** 013223
- [23] Shaydulin R and Pistoia M 2023 QAOA with $n \cdot p \geq 200$ 2023 *IEEE Int. Conf. on Quantum Computing and Engineering (QCE)* vol 01 pp 1074–7
- [24] Moses S A et al 2023 A race-track trapped-ion quantum processor *Phys. Rev. X* **13** 041052
- [25] DeCross M, Chertkov E, Kohagen M and Foss-Feig M 2023 Qubit-reuse compilation with mid-circuit measurement and reset *Phys. Rev. X* **13** 041057
- [26] Sciorilli M, Borges L, Patti T L, García-Martín D, Camilo G, Anandkumar A and Aolita L 2025 Towards large-scale quantum optimization solvers with few qubits *Nat. Commun.* **16** 476
- [27] Montañez-Barrera J A and Michielsen K 2025 Toward a linear-ramp qaoa protocol: evidence of a scaling advantage in solving some combinatorial optimization problems *npj Quantum Inf.* **11** 131
- [28] Montañez-Barrera J A, Michielsen K and Neira D E B 2025 Evaluating the performance of quantum processing units at large width and depth (arXiv:2502.06471)
- [29] Hauke P, Katzgraber H G, Lechner W, Nishimori H and Oliver W D 2020 Perspectives of quantum annealing: methods and implementations *Rep. Prog. Phys.* **83** 054401
- [30] Aharonov D, van Dam W, Kempe J, Landau Z, Lloyd S and Regev O 2007 Adiabatic quantum computation is equivalent to standard quantum computation *SIAM J. Comput.* **37** 166–94
- [31] Zbinden S, Bärttschi A, Djidjev H and Eidenbenz S 2020 Embedding algorithms for quantum annealers with chimera and pegasus connection topologies *High Performance Computing* ed P Sadayappan, B L Chamberlain, G Juckeland and H Ltaief (Springer) pp 187–206
- [32] Lopez-Bezanilla A, King A D, Nisoli C and Saxena A 2024 Quantum fluctuations drive nonmonotonic correlations in a qubit lattice *Nat. Commun.* **15** 589
- [33] Jünger M, Lobe E, Mutzel P, Reinelt G, Rendl F, Rinaldi G and Stollenwerk T 2021 Quantum annealing versus digital computing: an experimental comparison *ACM J. Exp. Algorithmics* **26** 1–30
- [34] Tasseff B, Albash T, Morrell Z, Vuffray M, Likhov A Y, Misra S and Coffrin C 2024 On the emerging potential of quantum annealing hardware for combinatorial optimization *J. Heuristics* **30** 325–58
- [35] Pelofske E, Bärttschi A and Eidenbenz S 2023 Quantum annealing vs. QAOA: 127 qubit higher-order ising problems on nisq computers *Int. Conf. on High Performance Computing* (Springer) pp 240–58
- [36] Pelofske E, Bärttschi A and Eidenbenz S 2024 Short-depth QAOA circuits and quantum annealing on higher-order ising models *npj Quantum Inf.* **10** 30
- [37] Lubinski T, Coffrin C, McGeoch C, Sathe P, Apanavicius J and Neira D E B 2024 Optimization applications as quantum performance benchmarks *ACM Trans. Quantum Comput.* **5** 1–44
- [38] Mohseni N, Morstyn T, Meara C O, Bucher D, Nüßlein J and Cortiana G 2025 Evidence of quantum scaling advantage in approximate optimization for energy coalition formation with 100+ agents *Quantum Sci. Technol.* **11** 015009
- [39] Commander C W 2009 *Maximum cut Problem (MAX-Cutmaximum Cut Problem, MAX-CUT)* (Springer) pp 1991–9

- [40] Mundada P S et al 2023 Experimental benchmarking of an automated deterministic error-suppression workflow for quantum algorithms *Phys. Rev. Appl.* **20** 024034
- [41] Lykov D, Wurtz J, Poole C, Saffman M, Noel T and Alexeev Y 2023 Sampling frequency thresholds for the quantum advantage of the quantum approximate optimization algorithm *npj Quantum Inf.* **9** 73
- [42] Egger D J, Mareček J and Woerner S 2021 Warm-starting quantum optimization *Quantum* **5** 479
- [43] Tate R, Moondra J, Gard B, Mohler G and Gupta S 2023 Warm-started QAOA with custom mixers provably converges and computationally beats goemans-williamson's max-cut at low circuit depths *Quantum* **7** 1121
- [44] Truger F, Barzen J, Bechtold M, Beisel M, Leymann F, Mandl A and Yussupov V 2023 Warm-starting and quantum computing: a systematic mapping study (arXiv:2303.06133)
- [45] Tate R, Farhadi M, Herold C, Mohler G and Gupta S 2023 Bridging classical and quantum with SDP initialized warm-starts for QAOA *ACM Trans. Quantum Comput.* **4** 1–39
- [46] Magann A B, Arenz C, Grace M D, Ho T-S, Kosut R L, McClean J R, Rabitz H A and Sarovar M 2021 From pulses to circuits and back again: a quantum optimal control perspective on variational quantum algorithms *PRX Quantum* **2** 010101
- [47] Magann A B, Rudinger K M, Grace M D and Sarovar M 2022 Lyapunov-control-inspired strategies for quantum combinatorial optimization *Phys. Rev. A* **106** 062414
- [48] Q-CTRL 2023 Q-CTRL Blogs: avoiding an unexpected roadblock in quantum computing - compilation (available at: <https://q-ctrl.com/blog/avoiding-an-unexpected-roadblock-in-quantum-computing-compilation>)
- [49] Kakkar A, Marsh S, Wang Y, Mundada P, Coote P, Hartnett G, Biercuk M J and Baum Y 2025 No need to calibrate: characterization and compilation for high-fidelity circuit execution using imperfect gates (arXiv:2511.21831)
- [50] Coote P, Dimov R, Maity S, Hartnett G S, Biercuk M J and Baum Y 2025 Resource-efficient context-aware dynamical decoupling embedding for arbitrary large-scale quantum algorithms *PRX Quantum* **6** 010332
- [51] Q-CTRL 2023 Q-CTRL documentation: algorithmic benchmarks (available at: <https://docs.q-ctrl.com/fire-opal/discover/adopt/algorithmic-benchmarking-results>)
- [52] Auger A and Hansen N 2005 A restart cma evolution strategy with increasing population size *2005 IEEE Congress on Evolutionary Computation* vol 2 pp 1769–76
- [53] Barkoutsos P K, Nannicini G, Robert A, Tavernelli I and Woerner S 2020 Improving variational quantum optimization using CVaR *Quantum* **4** 256
- [54] Zhou L, Wang S-T, Choi S, Pichler H and Lukin M D 2020 Quantum approximate optimization algorithm: performance, mechanism and implementation on near-term devices *Phys. Rev. X* **10** 021067
- [55] Akshay V, Rabinovich D, Campos E and Biamonte J 2021 Parameter concentrations in quantum approximate optimization *Phys. Rev. A* **104** L010401
- [56] Dam W, Eldefrawy K, Genise N and Parham N 2021 Quantum optimization heuristics with an application to knapsack problems *2021 IEEE Int. Conf. on Quantum Computing and Engineering (QCE)* (IEEE Computer Society) pp 160–70
- [57] Caha L, Kliesch A and Koenig R 2022 Twisted hybrid algorithms for combinatorial optimization *Quantum Sci. Technol.* **7** 045013
- [58] Sack S H, Medina R A, Kueng R and Serbyn M 2023 Recursive greedy initialization of the quantum approximate optimization algorithm with guaranteed improvement *Phys. Rev. A* **107** 062404
- [59] Dupont M, Sundar B, Evert B, Neira D E B, Peng Z, Jeffrey S and Hodson M J 2025 Benchmarking quantum optimization for the maximum-cut problem on a superconducting quantum computer *Phys. Rev. Appl.* **23** 014045
- [60] Wurtz J, Sack S and Wang S-T 2024 Solving non-native combinatorial optimization problems using hybrid quantum-classical algorithms (arXiv:2403.03153)
- [61] Pelofske E 2024 QAOA vs. QA (available at: https://github.com/lanl/QAOA_vs_QA)
- [62] Barahona F, Jünger M and Reinelt G 1989 Experiments in quadratic 0–1 programming *Math. Program.* **44** 127–37
- [63] Jumaa B R and Al-Jilawi A S 2020 Solving max-cut optimization problem *J. Phys.: Conf. Ser.* **1591** 012051
- [64] Shaydulin R and Pistoia M 2023 Data from “QAOA with $N \cdot p \geq 200$ ” *zenodo* (available at: <https://zenodo.org/records/7689982>)
- [65] Hagberg A A, Schult D A and Swart P J 2008 Exploring network structure, dynamics and function using networkx *Proc. of the 7th Python in Science Conf.*, ed G Varoquaux, T Vaught and J Millman pp 11–15
- [66] Pelofske E, Bärtschi A, Cincio L, Golden J and Eidenbenz S 2024 Scaling whole-chip QAOA for higher-order ising spin glass models on heavy-hex graphs *npj Quantum Inf.* **10** 132
- [67] Valiante E, Hernandez M, Barzegar A and Katzgraber H G 2021 Computational overhead of locality reduction in binary optimization problems *Comput. Phys. Commun.* **269** 108102
- [68] Mathieu C and Schudy W 2008 *Yet Another Algorithm for Dense max cut: go Greedy* pp 176–82
- [69] Costello K P, Shapira A and Tetali P 2011 Randomized greedy: new variants of some classic approximation algorithms *Proc. of the Twenty-Second Annual ACM-SIAM Symp. on Discrete Algorithms (SODA'11)* (Society for Industrial and Applied Mathematics) pp 647–55
- [70] Havlíček V, Córcoles A D, Temme K, Harrow A W, Kandala A, Chow J M and Gambetta J M 2019 Supervised learning with quantum-enhanced feature spaces *Nature* **567** 209–12
- [71] IBM Corporation 2024 IBM documentation (available at: <https://quantum.cloud.ibm.com/docs/en/api/qiskit/qiskit.circuit.library.ZZFeatureMap>)
- [72] D-Wave 2024 D-Wave systems
- [73] Wurtz J and Love P J 2021 Classically optimal variational quantum algorithms *IEEE Trans. Quantum Eng.* **2** 1–7
- [74] Amaro D, Modica C, Rosenkranz M, Fiorentini M, Benedetti M and Lubasch M 2022 Filtering variational quantum algorithms for combinatorial optimization *Quantum Sci. Technol.* **7** 015021
- [75] Cadavid A G, Dalal A, Simen A, Solano E and Hegade N N 2025 Bias-field digitized counterdiabatic quantum optimization *Phys. Rev. Research* **7** L022010
- [76] Granet E and Dreyer H 2025 Benchmarking a heuristic Floquet adiabatic algorithm for the Max-Cut problem *Sci. Rep.* **15** 31983
- [77] McGeoch C C, Chern K, Farré P, and King A K 2024 A comment on comparing optimization on D-Wave and IBM quantum processors
- [78] Pelofske E 2025 Comparing three generations of D-Wave quantum annealers for minor embedded combinatorial optimization problems *Quantum Sci. Technol.* **10** 025025
- [79] Azure Quantum 2024 Quantum provider
- [80] Hansen N 2016 The CMA evolution strategy: a tutorial (arXiv:1604.00772)